DTU Compute
Department of Applied Mathematics and Computer Science

Machine Learning for Semi-Automated Annotations of the Linear A Inscriptions

Bachelor Thesis

DTU Compute
Department of Applied Mathematics and Computer Science
Technical University of Denmark

Richard Petersens Plads
Building 324
2800 Kongens Lyngby, Denmark
www.compute.dtu.dk

Preface

This Bachelor's thesis was prepared at the Department of Applied Mathematics and Computer Science at the Technical University of Denmark in fulfillment of the requirements for acquiring a Bachelor of Science in Engineering (BSc Eng) degree in General Engineering (Cyber Systems).

Kongens Lyngby, May 27, 2026

Frederik W. L. Christoffersen

Abstract

This thesis is in the workings... [Write something when finished with all thesis chapters]

Acknowledgements

Frederik W. L. Christoffersen
General Engineering (Cyber Systems), DTU
Author of this thesis

Morten Mørup
Professor, DTU Compute
Main supervisor

Ester Salgarella
Research Fellow, AIAS Aarhus / University of Cambridge
Co-supervisor, creator of SigLA, and winner of the 2026 Michael Ventris Award in Mycenaean Studies

Lukas Esterle
Associate Professor, Aarhus University
Co-supervisor

Generative AI tools were occasionally used as support for brainstorming, linguistic editing, code development, and research during the development of this project.

Thanks to Morten for making this project possible. Thanks to Lukas for his tips and support in developing the fundamental idea for it. And many thanks to Ester for her positive attitude, the resources she provided, and the time she put into it. I am grateful for the opportunity to have worked with you.

Contents

Acronyms

AIAS Aarhus Institute of Advanced Studies

CannyFill Canny edge detection with morphological closing

Cefael Collections de l'École française d'Athènes en ligne

CNN Convolutional Neural Network

CV Computer Vision

Dice Dice-Sørensen coefficient

FATESAM Few-shot Adaptation of Training frEe SAM

GAN Generative Adversarial Network

Gaussian Gaussian adaptive thresholding

GFSAM Graph-based Few-shot Segment Anything Semantically

GORILA Godart and Olivier, Recueil des inscriptions en linéaire A

HT Haghia Triada Tablet

ICL In-Context Learning

IPR Intellectual Property Rights

IoU Intersection over Union

LLM Large Language Model

ML Machine Learning

MLP Multilayer Perceptron

MVP Minimum Viable Product

NLP Natural Language Processing

Otsu Otsu's method

PDR Project Definition Report

POI Point of Interest

RQ Research Question

SA-1B Segment Anything 1 Billion

SAM Segment Anything Model

SAM+pts SAM with automatic point prompts

SOTA State of the Art

TPE Tree-structured Parzen Estimator

ViT Vision Transformer

YOLO You Only Look Once

List of Figures

List of Tables

CHAPTER 1

Introduction

1.1 Background

When the British archaeologist Arthur Evans in the beginning of the 20th century went to excavate Knossos on Crete, he found a huge amount of clay tablets written in a previously unknown linear script. Some documents were clearly of a newer script, which he called *Linear B*, while others were of an older one, which he named *Linear A*. Some fifty years later, the British amateur archaeologist Michael Ventris finally succeeded in deciphering the newer script Linear B, which he proved to be representing Mycenaean Greek, the earliest attested form of Greek. But the same hypothesis did not hold for the older script Linear A, whose encoded language remained unknown, still yet to be deciphered.

A key obstacle to its decipherment has been the small size of its corpus, lately consisting of only around 2,500 inscriptions (about 1,400 if only including those well-preserved enough to be readable) - significantly smaller than that of its younger counterpart that now spans more than 4,500 inscriptions (which are comparatively longer and better preserved).[1,2] But new inscriptions are still being discovered, with the corpus most recently extended in 2025 by over 100 new documents.[3,4]

Another challenge has been the lack of digital representations of these inscriptions, in a form suitable for statistical analysis. While photographs with accompanied by expert drawings of nearly the entire corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA) - are available online through Collections de l'École française d'Athènes en ligne

[1] Salgarella 2020.
[2] Salgarella 2025, p. 13, 46.
[3] Del Freo and Zurbach 2025.
[4] Consiglio Nazionale delle Ricerche 2025.

(Cefael) (see Godart and Olivier (1976-1985)), these are only available in print form. This motivated the development of the palæographical[5] database *The Signs of Linear A* (SigLA), which seeks to "[develop] a systematic, exhaustive and user-friendly open access database of all Linear A inscriptions [...] in order to carry out statistical and palæographic analyses within the epigraphic corpus" (Salgarella and Castellan 2020). However, the last addition to SigLA was in 2021, and the project has so far managed to document just 772 of all currently known inscriptions.[6,7]

1.1.1 A problem of manual digitization

The drawings currently stored on SigLA were made by Ester Salgarella using the painting tool Krita. For each document, she manually created her own annotated drawings based on the GORILA originals stored by Cefael as raw tablet images accompanied by expert drawings. From these, she proceeded to create layered Krita files with metadata for each sign, which could then at last be exported as JSON files to SigLA.[8] According to her (personal communication), this has been a highly time-consuming undertaking. And the necessity of this kind of labor-intensive work has arguably been the biggest reason to why SigLA still does not contain the entire available corpus.

Due to the potential violation of Intellectual Property Rights (IPR), it is also of particular interest, whether this process can begin from the digital source images themselves, before relying on already completed but undigitized expert drawings as input. The aim of this project was therefore to investigate whether parts of the process can be automated by working on the raw tablet images from GORILA. More specifically, whether some kind of semi-automatic segmentation tool (given just the raw tablet images as input) can accelerate the workflow, while maintaining high-quality outputs that are still compatible with SigLA's database.

[5]The term "palæographical" refers to the study of ancient writing systems.
[6]SigLA 2021a.
[7]SigLA 2021b.
[8]Salgarella and Castellan 2020.

1.1.2 A problem of small data

The small size of the Linear A corpus and the irregularities in quality of its documents is a huge problem for the application of deep learning approaches, which typically require large amounts of data to perform well. Consequently, this project instead sought to make use of generalized Machine Learning (ML) and Computer Vision (CV). In particular Meta's State of the Art (SOTA) interactive segmentation model, the Segment Anything Model (SAM) (Kirillov et al. 2023), which is pre-trained on a massive and diverse dataset - Segment Anything 1 Billion (SA-1B), of over 1 billion masks. SAM's generalization and interactivity (taking user-placed prompts that guide the segmentation) potentially allows for semi-automatic annotation of Linear A inscriptions without the need for large amounts of training data, which was unfortunately not available in this context. It was therefore of particular interest to map the capabilities and limitations of SAM within this domain.

1.2 Related work

Within the last decade, the field of document annotation has seen significant advancements through the use of machine learning. Take for example the EU-funded generic CNN-based framework for historical document processing *dhSegment* (Oliveira, Seguin, and Kaplan 2018). Or the more recent work on segmentation of stone inscriptions (P. Zhang, C. Li, and Sun 2024), where SAM and other state-of-the-art methods are beaten as benchmarks by their stacked U-Net and GAN architecture. Or even the YOLO and Roboflow based approach to segmentation of Palmyrene Aramaic inscriptions (Hamplová et al. 2024) that augment[9] their dataset with a Generative Adversarial Network (GAN) to increase performance.

Nevertheless, these methods all depend on deep learning, making them inconvenient for domains of small data, which instead seem to require a generalized and interactive approach such as SAM.

[9]The term *augmenting* refers to the process of increasing the size of a dataset.

1.2.1 SAM and its applications

Ever since its release in 2023, SAM has been extensively evaluated across a wide range of CV tasks to investigate whether it can really "segment anything". Papers have evaluated its performance within niche domains, such as medical images (Ma et al. January 2024), camouflaged objects (Tang, Xiao, and B. Li 2023), transparent objects (Han et al. 2023), hierarchical texts (Ye et al. 2024), and even oracle bone inscriptions (Wang et al. 2025). Most of these papers found that SAM still tends to struggle with such domains, despite its impressive zero-shot[10] generalization. Meanwhile, some of them also found that this performance can be significantly improved through architectural extensions and domain-specific fine-tuning, which unfortunately requires deep learning and a lot of data. However, equally, it has been shown that SAM's performance can significantly improve through effective prompt engineering alone (Price, Rundensteiner, and Cote 2025), requiring no deep learning at all. And within the domain of 3D medical images, a training-free few-shot adaptation of SAM2 (see Ravi et al. 2024) has been recently demonstrated to surpass supervised methods like U-Net, and even the best fined-tuned versions of SAM (He et al. 2025b). Thus, it seemed reasonable to test out the limits of SAM's generalization and interactivity within the domain of Linear A document images, to investigate whether it could be used to accelerate the annotation workflow of Ester Salgarella.

1.3 Problem statement

As previously stated, the aim of this project was to investigate whether parts of the process of creating annotated drawings like the ones in SigLA can be automated by working on the raw tablet images from GORILA. More specifically, whether some kind of semi-automatic segmentation tool (given just the raw tablet images as input) using interactive and generalized CV - specifically the Segment Anything Model (SAM) - can accelerate the workflow, while maintaining high-quality outputs that are still compatible with a database like SigLA. But what would be the

[10]The term *zeroshot* refers to model performance on hitherto unseen data.

relative segmentation quality of such a tool? How interaction efficient would it be exactly? And by how much could it improve the efficiency of the workflow? To concretize the research, the problem statement was decomposed into three Research Questions (RQs) - each accompanied by their respective operationalization (see the following subsection for details).

1.3.1 Research questions

1. **RQ1 (Segmentation quality)**: *How well does a SAM-based segmentation approach perform quantitatively in terms of segmentation accuracy, compared to simple classical image processing baselines when applied to Linear A document images?*

   Operationalization: SAM-based masks were compared to classical image-processing baselines using overlap metrics (i.e. Dice) against the ground truth segmentations derived from SigLA's Krita files using a self-made dataset of 60 documents.

2. **RQ2 (Interaction efficiency)**: *To which extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs?*

   Operationalization: Two self-developed segmentation tools - one SAM-based, one classical - were evaluated in a case study of 3 documents. The time spent in each tool by Ester Salgarella from image import to mask export was compared against the segmentation quality (i.e. Dice) relative to ground truth (SigLA's Krita files).

3. **RQ3 (Workflow efficiency)**: *How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions?*

   Operationalization: The time it took Ester Salgarella to annotate manually compared to being assisted by two self-developed segmentation tools - one based on SAM, and the other based on a classical baseline - was measured in a case study of 3 documents.

1.4 Impact – innovation and application

The work of this thesis presents itself as a first proof-of-concept (and a mapping of the difficulties involved) in using SAM and modern CV to accelerate the annotation of Linear A documents. Its main contribution is a prototype of a semi-automatic annotation tool that extracts engravings from raw tablet images, along with an evaluation framework that can be used to assess the performance of that tool and any potential future derivatives. Such a tool could potentially also be applied to Linear B, as it is the whole field of Aegean studies that seems to suffer from severe under-digitization of the primary evidence (which is a huge problem for the scientific investigations of scholars). Hopefully, the work will inspire further research and development within the field of digital epigraphy. Particularly in the application of generalized and interactive CV approaches such as SAM on Linear A, and perhaps even in the broader context of use on small corpus ancient writing systems, where the lack of big data is ill fit for approaches using deep learning.

1.5 Overview of the thesis

The rest of this thesis is structured into four main chapters (Data, Methodology, Results and Discussion, Conclusion), all split up into sections concerning each of the three research questions.

CHAPTER 2

Data

2.1 Sources

The dataset used in this thesis was constructed from 60 images of raw Linear A inscriptions, and their corresponding ground truth annotations. Cefael (see Godart and Olivier 1976-1985) was chosen as the source for raw document images, since it is the only online and widely publicly available source of the printed five volume corpus - Godart and Olivier, Recueil des inscriptions en linéaire A (GORILA). As ground truth, annotations provided by Ester Salgarella from SigLA (see Salgarella and Castellan 2020) were chosen instead of the annotated drawings from the corpus, as the former are more up to date, and of better quality.

What follows in this chapter is a detailed description of this dataset, documenting its characteristics, its construction process, and its limitations and biases.

2.2 Characteristics and construction

The dataset was made to consists of 60 chosen image pairs of raw documents and their corresponding ground truth annotations. Each of these image pairs was categorized into one of three difficulty levels (easy, medium, hard) based on the visual quality of the inscriptions (see Section 2.2.1). They were then split into a test set of 27 images, a validation set of 27 images, and a support set of 6 images, with each set containing an equal proportion of images from each of the three difficulty levels.

Only inscriptions with clay as writing support were included, as they are the most common and representative of the Linear A corpus, and were the only ones for which SigLA ground truth annotations were al-

ready available. The raw document images from Cefael are screenshots of grayscale images placed within the scanned printed edition of the corpus, and are thus not of the highest quality, being the only widely publicly available images of the Linear A inscriptions. For an example of a raw document image and its corresponding ground truth annotation, see Figure 2.1.

((a)) Raw image (HT7b) **((b))** Ground truth (HT7b)

Figure 2.1: Example of a raw inscription image and its corresponding ground truth annotation.

2.2.1 Difficulty classification

The dataset was categorized into three difficulty levels (easy, medium, hard) based on the visual quality of the raw images and the clarity of the inscriptions. This classification was performed manually by inspecting each image and considering factors such as contrast, noise, and the presence of tablet breakages or general damage to the inscriptions. The concrete criteria used to classify the difficulty sets were as follows:

- **Easy:** labeled as such for documents with clear ingravings, good contrast, minimal noise, and constant lighting.
- **Hard:** labeled to those with poor contrast, significant noise, and substantial damage making it hard even for the naked eye to read the inscriptions.
- **Medium:** labeled to all those inbetween the qualities of easy and hard.

For visual examples of images within each of these difficulties, see Figure 2.2.

2.2.2 Image registration and preprocessing

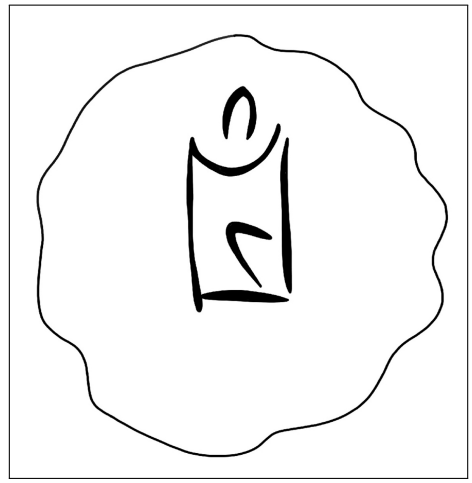
To obtain any kind of meaningful comparison between model outputs and ground truth, the raw images and their ground truth annotations had to be aligned and registered to each other. This was done manually in Krita for all 60 image pairs by downscaling the ground truth images to the individual scales of their raw counterparts, and aligning by translation and rotation, using the raw images as background reference.

2.3 Limitations and biases

2.3.1 Quality of the raw tablet images

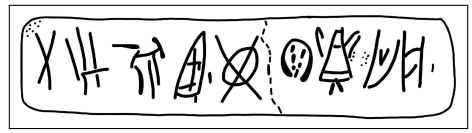
Due to the weak quality of the raw tablet images, they carry only parts of the visual information of the tablets they represent. When the drawings of GORILA were made, the researchers had access to the tablets themselves, meaning they could see what the images did not pick up, and better discern actual engravings from mere damages to the tablets. Therefore, these tablet images are far from the best source of data there is, being however, the only source publicly available. That lack of good quality data can unfortunately limits the output quality of segmentations constructed therefrom.

((a)) Easy KHWc2104 (raw)

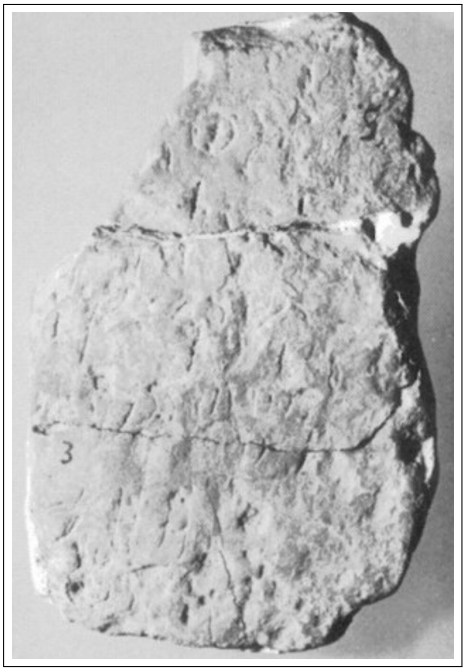


((b)) Easy KHWc2104 (gt)

((c)) Medium MA1a (raw)



((d)) Medium MA1a (gt)



((e)) Hard HT3 (raw)

((f)) Hard HT3 (ground truth)

Figure 2.2: Examples of images with different levels of segmentation difficulty: easy, medium, and hard. For each case, the raw image (left) and the corresponding ground truth annotation (right) are shown.

2.3.2 Alignment to the ground truth

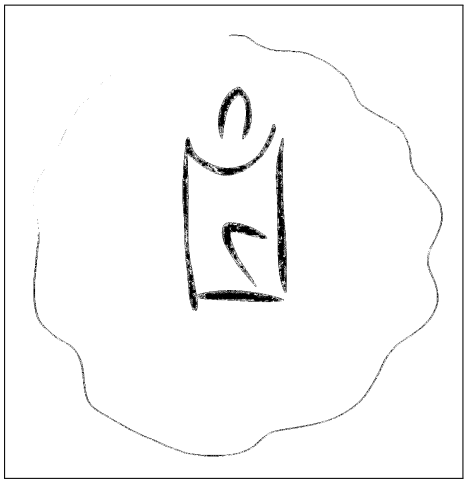
Even after image registration, ground truth images still did not perfectly align with the raw images, affecting the obtained segmentation quality. This misalignment was a source of noise in the evaluation. Ester's annotations were never constructed by overlaying them on the raw images, but rather by doing so on the accompanied corpus drawings, only using the raw images as reference. Thus, the raw images and the ground truth annotations were never perfectly aligned to begin with, and the manual registration process could only do so much to mitigate that. This is seen by the low Dice values obtained for all models in the results (see Section 4.1).

2.3.3 Binarization of the drawings

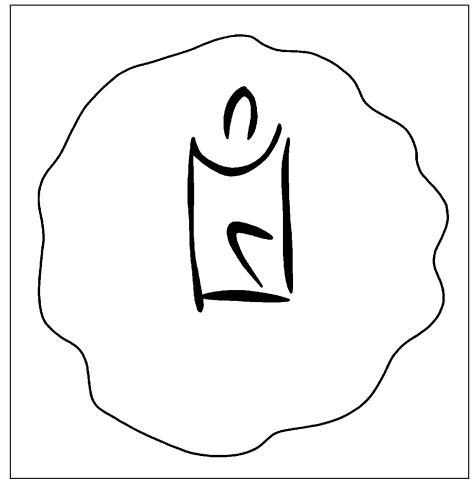
Due to a missing threshold step of the registered JPEGs in the initial registration of the dataset, the ground truth annotations were to begin with binarized with a threshold of 0 instead of 128, meaning that only the pure black pixels of the drawings were considered as part of the ground truth foreground, while the lighter gray pixels were not accounted for. This led to lower Dice scores in general, as the ground truth in practice was thinner and more sparse compared to the actual drawings.

This issue was for the evaluation later corrected to a threshold of 128, meaning that all pixels darker than 128 now were considered foreground, and all pixels lighter than that were considered background (see Figure 2.3 for visual examples of the two different ways of binarization). However, only the hyperparameter optimization of the models was not redone for the new binarization format, and was thus performed with ground truth in the old binarization format.

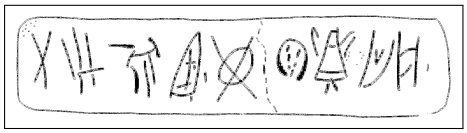
Arguably, this only affected the effectiveness of the optimization and what the models were optimized for (penalizing non-conservative segmentation styles more than otherwise), while still preserving fairness and generalizability, as all model hyperparameters were tuned under the same conditions.



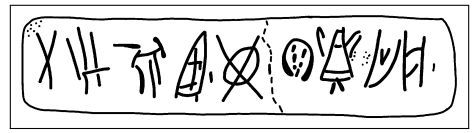
((a)) KHWc2104 (old)



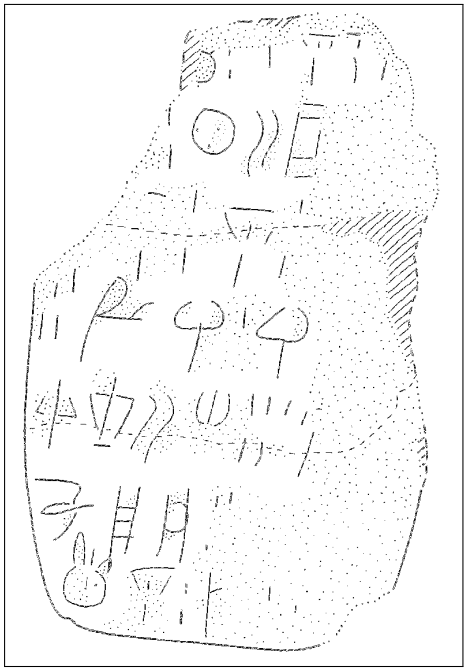
((b)) KHWc2104 (new)



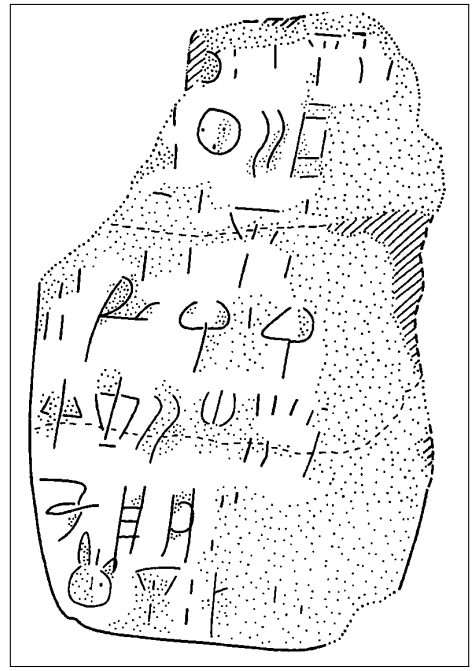
((c)) MA1a (old)



((d)) MA1a (new)



((e)) HT3 (old)



((f)) HT3 (new)

Figure 2.3: Examples of images processed with the two different binarization thresholds; 0 (old) and 128 (new).

CHAPTER 3
Methodology

This chapter describes in detail the operationalization of answering the question of whether a SAM-based segmentation tool can accelerate the annotation workflow of Linear A documents (see Section 1.3.1 for an overview of the individual RQs). Overall, this operationalization was achieved through a comprehensive evaluation of segmentation quality (see Section 3.1), and a case study involving Dr. Ester Salgarella measuring the interaction efficiency (see Section 3.2) and workflow efficiency (see Section 3.3) of a self-developed SAM-based segmentation tool. In that way, these three abstract virtues were decomposed into independent measurable metrics, making it possible to concretely answer the research questions, and where the potential benefits and drawbacks of SAM lie.

3.0.1 Development environment and tools

All software written for this thesis was made publicly available on GitHub (Christoffersen 2026). Python 3.13 was used for all code development except frontend. The choice of this programming language was made primarily due to it being the language of choice for the ML and SAM implementation libraries needed. Its big community, and library support, and package manager (pip) also made it exceptionally easy to install and manage dependencies, and set up the development environment. The particular version was chosen, as it was the second latest stable release at the time of development, with the risk of Python 3.14 still not having full support by all libraries.

3.1 Non-interactive segmentation ability

To answer the question of RQ1: *How does the initial segmentation quality of SAM compare to that of classic segmentation methods?*, the initial fully-automatic segmentation quality of SAM was evaluated against a set of classical baseline models, using a suitable segmentation metric (i.e. Dice score) computed from the pairs of predicted and ground truth masks. All models were tuned by hyperparameter optimization to their best performance on the dataset, and the best performing version of SAM was compared against the best performing baseline model.

The following subsections therefore give an explanation of the segmentation problem itself and document the configuration of the baseline and state-of-the-art models, as well as the evaluation protocol used to answer RQ1.

3.1.1 The problem in mathematical terms

The task of extracting the foreground (engraving and outline) from the background of an image of a writing support can be conceptualized as a binary image segmentation and classification problem at the pixel level, where each pixel is classified as either belonging to the foreground or the background. Although this segmentation is binary, it is single-class in the sense that all foreground pixels are treated as a single foreground class, without distinguishing between different types of foreground, like tablet outlines, sign types, etc. This segmentation process can be mathematically described as a mapping that - given some predicate function f, and an input image I - produces a binary mask B of the same dimensions, where each pixel value indicates whether it belongs to the foreground (by convention denoted as 1) or the background (denoted as 0):

$$B(x,y) = \begin{cases} 1 & \text{if } f(I,x,y) \\ 0 & \text{otherwise} \end{cases}$$

In a threshold-based approach for example, the predicate function can be expressed as $f(I,x,y) = I(x,y) > T$, where T is the threshold value

that determines the cutoff between foreground and background pixels.[1] The predicate function does not, however, have to depend only on the individual pixel value being classified. It can for example also be defined based on an attribute computed from the global image context (see global thresholding with Otsu in Section 3.1.2.2). Or it can be determined from the local context of the pixel neighborhood (see local adaptive thresholding in Section 3.1.2.3), which can be expressed by kernel convolution working on neighborhood intensities of a window $w \in \mathbb{R}^{m \times n}$ around each pixel (here $a = (m-1)/2$ and $b = (n-1)/2$ are the vertical and horizontal half-widths of w, respectively, assuming odd dimensions):[2,3]

$$(w * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} w(i, j) I(x - i, y - j)$$

Even the entire image can be given as context with external pointers, as in the case of SAM, where the predicate function can be understood as a learned, non-linear black-box function that takes both image and prompts to produce the binary output mask. Thus, the segmentation problem could evidently be approached by many different methods, each with their own contextual arguments and inner workings f. And of those evaluated, it was just a matter of choosing the best one (for the evaluation details used to do that, see Section 3.1.4).

3.1.2 Baseline model configuration

The classical baseline models against which SAM was evaluated (from simplest to most advanced) were:

- Global thresholding (Otsu's method)
- Local adaptive thresholding (Gaussian)
- Edge-based segmentation (Canny edge detection with region filling)

[1] Gonzalez and Woods 2018, p. 743.
[2] Gonzalez and Woods 2018, p. 159.
[3] Gonzalez and Woods 2018, p. 157.

Together, they cover a range of classic segmentation techniques that might be used for binary foreground extraction. Besides a description of the pre-filtering method used for all of them, what follows is a brief description of each of these methods, including their characteristics, how they work, and their implementation details.

3.1.2.1 Bilateral pre-filtering

As a pre-filtering step for all the classical models, a bilateral filter (see Tomasi and Manduchi 1998) was applied to the input image before feeding it to the classical methods. In this way, more noise could be reduced while preserving engravings, as this filter is known to be effective at blurring out noise while preserving edges thanks to its combined distance and intensity-dependent weighting. Discretely (see Pan 2025), its output value $I_{\mathrm{bf}}(p)$ at a given two-dimensional pixel position p (where q is the two-dimensional position of one of its neighbors within the kernel window with size d) can be defined as follows:

$$I_{\mathrm{bf}}(p) = \frac{\sum_q I(q)\, W_{\mathrm{space}}(p,q)\, W_{\mathrm{color}}(p,q)}{\sum_q W_{\mathrm{space}}(p,q)\, W_{\mathrm{color}}(p,q)}$$

Here, the spatial weight based on the distance between pixels p and q, and the color range weight based on the intensity difference between p and q are respectively given by:

$$W_{\mathrm{space}}(p,q) = \exp\left(-\frac{\|p-q\|^2}{2\sigma_{\mathrm{space}}^2}\right)$$

$$W_{\mathrm{color}}(p,q) = \exp\left(-\frac{\|I(p)-I(q)\|^2}{2\sigma_{\mathrm{color}}^2}\right)$$

It thus takes three hyperparameters: the kernel window diameter d, the spatial standard deviation σ_{space}, and the intensity standard deviation σ_{color}. However, inspired by (OpenCV 2026), it was decided to use the same σ value for both spatial and color weights for the implementation, as this simplified the hyperparameter tuning (see Section 3.1.4.2) without significantly compromising the filter's performance.

3.1.2.2 Global thresholding (Otsu's method)

Otsu's method is a global thresholding method that works on gray-level histograms, which was originally proposed by Otsu (1979). As the simplest of the baselines, it represents the minimum acceptable segmentation quality, since it only takes the global gray level distribution into account, with no assumptions about any spatial relationships between pixels. It is therefore expected to perform poorly on images with erosion, uneven lighting, and differences in surface texture[4], which are the exact characteristics of the images to be segmented.

Here is how it works: First, let pixels of the raw image be divided into L shades of gray $[1, 2, ..., L]$. Then, let each pixel be assigned one of two classes C_0 or C_1, guided by a threshold T, where C_0 contains the shades $[1, 2, ..., T]$, and C_1 contains the rest $[T + 1, T + 2, ..., L]$. Otsu's method then works by performing an exhaustive search over all L gray levels to find an optimal threshold T^* that results in a maximum between-class variance σ_B^2:

$$T^* = \underset{1 \leq T < L}{\arg\max}\ \sigma_B^2(T)$$

The between-class variance σ_B^2 is then defined by a function of the respective total probabilities of the two classes (ω_0 and ω_1) and the mean gray level value per class (μ_0 and μ_1):

$$\sigma_B^2(T) = \omega_0(T)\omega_1(T)[\mu_1(T) - \mu_0(T)]^2$$

3.1.2.3 Local adaptive thresholding (Gaussian)

Local adaptive thresholding works by computing a threshold for each pixel based on its local neighborhood. This is an advantage in cases where the image has varying light conditions (which is the case for the images to be segmented), as the threshold adapts to local lighting.

Mathematically, the local threshold T is a function of a given pixel position (x, y) computed by a weighted sum of the neighborhood. The

[4] Gonzalez and Woods 2018, p. 751.

generated binary mask B is then defined as:[5]

$$B(x,y) = \begin{cases} 1 & \text{if } I(x,y) > T(x,y) \\ 0 & \text{otherwise} \end{cases}$$

To compute $T(x,y)$, there are a few weighting functions to choose from. For this baseline, the Gaussian kernel was used, as it can limit the influence of distant noisy pixels by assigning weights based on distance from the kernel center. With a Gaussian kernel G_σ, the threshold $T(x,y)$ is computed as a Gaussian-weighted sum of local intensities subtracted by a constant C. Using kernel convolution ($*$), this can be expressed as:[6]

$$T(x,y) = (G_\sigma * I)(x,y) - C$$

In general, the weight at an offset (i,j) from the kernel center can be written as follows (where α is a normalization constant ensuring that the whole kernel sums to 1, and σ controls the spatial spread of the kernel):[7]

$$G_\sigma(i,j) = \alpha \cdot \exp\left(-\frac{i^2+j^2}{2\sigma^2}\right)$$

This gave in total 4 hyperparameters for this baseline (see Figure A.1 for a list of these):

3.1.2.4 Edge-based segmentation (Canny edge detection with region filling)

Canny edge detection is a method that by itself only segments the edges of an image, excluding the enclosed regions within those edges. First, for the image gradient of each pixel, the magnitude $M_s(x,y)$ and the orientation $\theta_s(x,y)$ are computed:[8]

$$M_s(x,y) = \| \nabla I_s(x,y) \| = \sqrt{\left(\frac{\partial I_s}{\partial x}\right)^2 + \left(\frac{\partial I_s}{\partial y}\right)^2}$$

[5] Gonzalez and Woods 2018, p. 761.
[6] OpenCV 2024.
[7] Gonzalez and Woods 2018, p. 167.
[8] Gonzalez and Woods 2018, p. 730.

$$\theta_s(x, y) = \arctan\left(\frac{\partial I_s/\partial y}{\partial I_s/\partial x}\right)$$

Next, *non-maxima suppression*[9] is applied to retain only local maxima of $M_s(x, y)$ along the gradient direction $\theta_s(x, y)$, resulting in thinner edges. This is then followed by *hysteresis thresholding*[10], classifying pixels as strong, weak, or non-edges with a low threshold T_l and a high threshold T_h, only preserving weak edges if connected to strong edges.

Inspired by (Rosebrock 2015), for the implementation, these thresholds were automatically determined by setting

$$T_l = \max\Big(0, (1-\sigma)\, v\Big), \quad T_h = \min\Big(255, (1+\sigma)\, v\Big)$$

turning σ into a hyperparameter that together with the grayscale median of the given image v controlled the sensitivity of the edge detection.

In an attempt to fill the regions enclosed by the detected edges, morphological closing (dilation followed by erosion) was applied to the edge mask. This gave in total 2 hyperparameters (see Figure A.2 for a list of these).

3.1.3 State-of-the-art

To be compared against the classical baselines was the generalized deep learning Segment Anything Model (SAM) (see Kirillov et al. 2023). Due to its popularity as a SOTA segmentation model, and the fact that its different use cases require different trade-offs in terms of accuracy, speed, and domain adaptation, there are many different variants of SAM.

For the sake of fairness, it seemed only reasonable to test out a range of them, including of course the original, the computationally efficient MobileSAM (see Zhao et al. 2023), and the newer SAM2 (see Ravi et al. 2024). [11] Additionally, a small ablation study was conducted on these base models to investigate whether these benefitted from the use of bilateral pre-filtering (see **??**) of the input image, and or automatic threshold-derived point prompts (see Section 3.1.3.5). Furthermore, due to their

[9] Gonzalez and Woods 2018, p. 730-731.

[10] Gonzalez and Woods 2018, p. 732.

[11] Due to access restrictions, the checkpoints (weight files) necessary for running the newly released SAM3 (Carion et al. 2025), it was not included in the evaluation.

potential of being better at accommodating to niche domains, In-Context Learning (ICL) implementations of SAM - like Few-shot Adaptation of Training frEe SAM (FATESAM), and Graph-based Few-shot Segment Anything Semantically (GFSAM) - were also included in the evaluation.

To be able to run all these computationally demanding variants of SAM, model execution was outsourced to Modal (see Modal Labs 2026) - a serverless cloud computing platform usually used for ML inference, tuning and training.

The following subsections describe the architecture of the chosen variants, and how they were implemented in the evaluation.

3.1.3.1 SAM architecture

For the original SAM and MobileSAM, the model architecture consists of an image encoder, and a comparatively lightweight prompt-guided mask decoder (see Figure 3.1). The image encoder used is a Vision Transformer (ViT) whose purpose is similar to that of a Convolutional Neural Network (CNN), processing an input image to extract its features. But instead of convolutional layers, a ViT is a transformer-based encoder traditionally used in Natural Language Processing (NLP) and Large Language Models (LLMs). As an alternative to words, it treats an image as a sequence of fixed size patches (regions of the image) embedded into vector representations with positional encodings, similar to how tokens work in NLP (see Figure 3.2). These embedded patches are then processed through a repeated series of normalization, feed-forward Multilayer Perceptrons (MLPs), and in-between those, a mechanism known as self-attention that weighs the patches according to their relevance to eachother.[12]

More specifically, given the input embedding X, self-attention computes the projections; quieries: Q, keys: K, and values: V, as products of X with respective learned matrices, and passes the information on as a new matrix (where d_k is the dimension of Q and K):

$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

[12] see Vaswani et al. 2023, for the math behind *attention*.

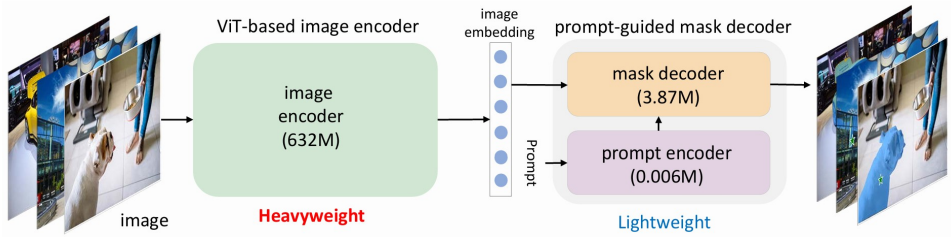


Figure 3.1: Architecture of SAM, from C. Zhang et al. (2023).

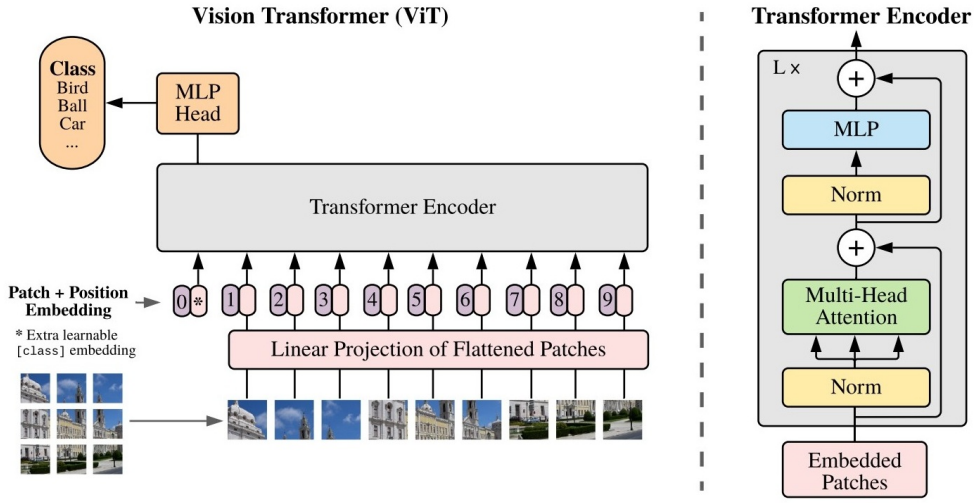


Figure 3.2: Architecture of the ViT, from Dosovitskiy et al. (2021).

Here, the softmax function normalizes the output to a probability distribution (putting each element on the interval $]0, 1[$, with their sum being 1), which is then multiplied by V (again turned into logits / non-normalized probabilities).

Thanks to this attention mechanism, after a number of these series-layers, the ViT finally outputs an image embedding containing the spatial context of the image.[13] It is this image embedding, combined with embedded prompts from the prompt encoder (referred to as prompt tokens) that are fed to the mask decoder to generate the final segmentation mask. However, before this happens, the prompt encoder takes any

[13] see Dosovitskiy et al. 2021, for how ViTs work in general.

given sparse prompts (i.e. box or point prompts) as embeddings by summing their positional encodings with learned type-specifc embeddings (one foreground/background embedding per point prompt, and two opposite corner embeddings per box prompt).

These are then fed to the lightweight mask decoder (which is also transformer-based) along with the image-embedding from the ViT, and 3 learnable output token embeddings (see Figure 3.3). Self-attention is applied to the prompt and output tokens, and two-way cross-attention (same as self-attention but with one embedding as queries, and the other as keys and values, and vice-versa) is applied between the tokens and image embedding for them to influence and attend one another. To keep track of spatial coordinates and prompts, positional encodings and the original prompt embeddings are added as input at every attention layer.

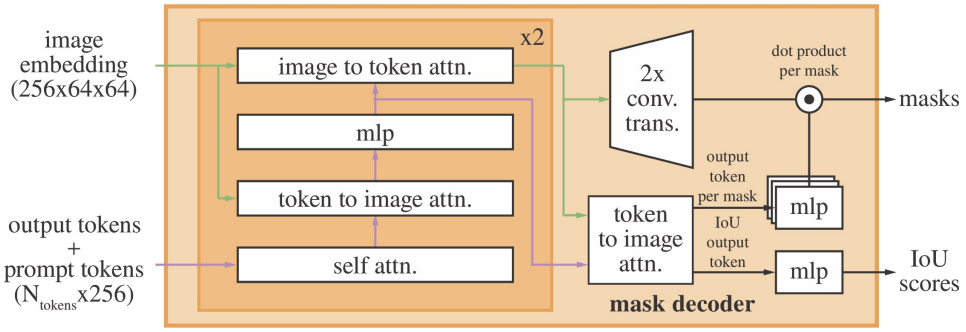


Figure 3.3: Architecture of SAM's mask decoder, from Kirillov et al. (2023).

At the end of this process, the image embedding is upsampled by convolution to the original image resolution. The three output tokens are then processed in parallel by two separate MLPs: one that generates their final segmentation masks by a dot product with the output and the upscaled image embedding, and another one that outputs a predicted IoU score for each mask. For single output masks, the mask is generated from a fourth output token.[14]

As for the implementation of SAM in the evaluation, the checkpoint used was ViT-H, since it is the biggest and best performing one. And

[14]Kirillov et al. 2023, p. 17.

as per definition, the MobileSAM implementation used the lighter ViT implementation TinyViT (Wu et al. 2022) with only 5M parameters compared to those of SAM (ViT-H, ViT-L, ViT-B with 632M, 307M, and 86M, respectively) - which is why it is computationally faster than the original.

3.1.3.2 SAM2

With the release of SAM2 (Ravi et al. 2024), the image encoder was changed to Hiera (Ryali et al. 2023) - a hierarchical ViT, which is more parameter efficient than vanilla ViTs, as tokens are pooled together at different stages of the encoding process, lowering attention complexity (see Figure 3.4). Additionally, a memory attention mechanism was added for consistency, allowing the model to attend to segmentation outputs of previous frames, making masks between frames consistent enough to segment videos. It does so by storing a memory-encoded version of the mask output of each previous frame in a memory bank that are then attended to by the image encoded embeddings before being passed to the mask decoder (see Figure 3.5).

The mask decoder did not change much from the original, except for the passing of early visual features (stride 4 and stride 8) from the Hiera image encoder to the convolutional upsampling process of the mask decoder, improving the integrity of the output with the injection of high quality spatial context (see Figure 3.6). However, also added were output markers for video tracking. More specifically, occlusion scores were implemented to determine whether an object of interest is out of frame or not. And object pointers were added as lightweight vectors each keeping track of the semantic information of each object of interest. And these object pointers are also stored in the memory bank together with the spatial memory, and attended to in memory attention.

As for the implementation of SAM2 in the evaluation, the checkpoint chosen was `sam2.1_hiera_large`, as it was the best performing one listed in the repository of SAM2 (Meta AI Research 2024). SAM2 also takes a checkpoint-specific YAML configuration file, which stores the hyperparameters for the model, and allows for easy overrides through Hydra.

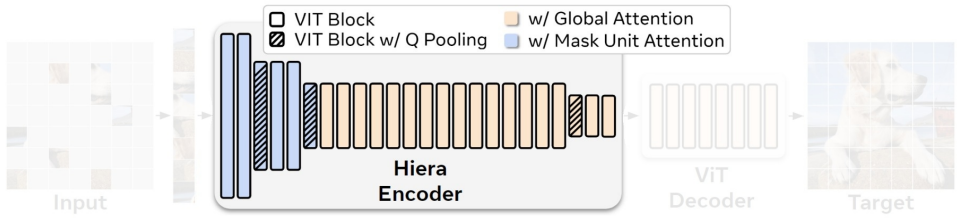


Figure 3.4: Architecture of Hiera, adapted from Ryali et al. (2023).

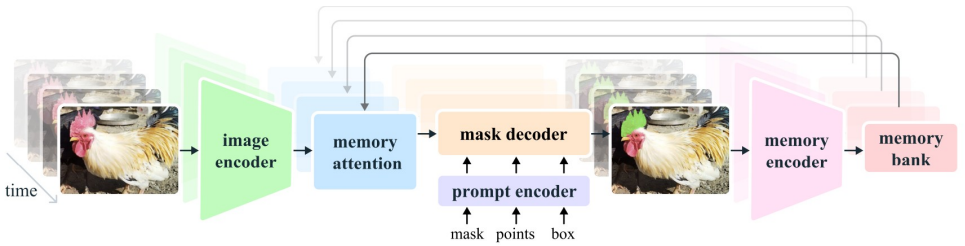


Figure 3.5: Architecture of SAM2, from Ravi et al. (2024).

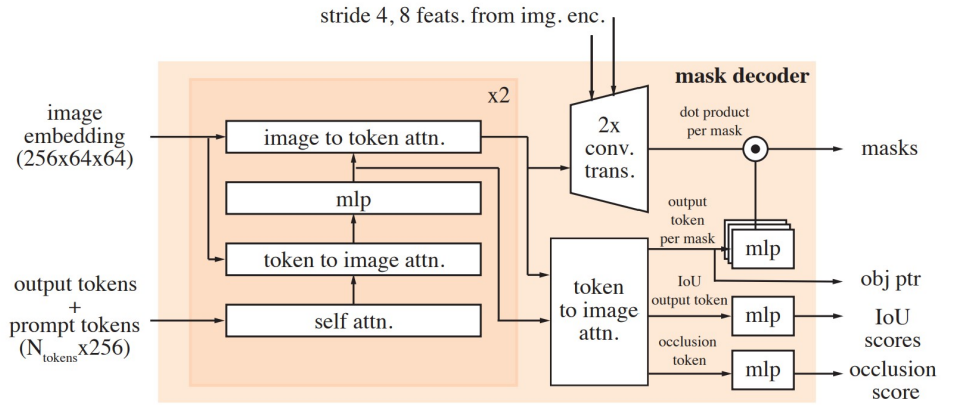


Figure 3.6: Architecture of SAM2's mask decoder, from Ravi et al. (2024).

3.1.3.3 FATESAM

Within the domain of 3D medical images, FATESAM - a training-free few-shot adaptation of SAM2 (see He et al. 2025b) has been recently demonstrated to surpass even the best fined-tuned versions of SAM. It works in principle by making use of SAM2's memory attention mechanism, by passing the few-shot support images to its memory encoder (see Figure 3.7), such that the mask decoder ends up with context to generate similar segmentation masks for the input query image.

These support images are chosen by a ranking of feature similarity to the input to make sure they are as representative as possible. This ranking is computed ascendingly from the Manhattan distances between the image-encoded features of the input and the support images, extracted from the Hiera image encoder of SAM2.

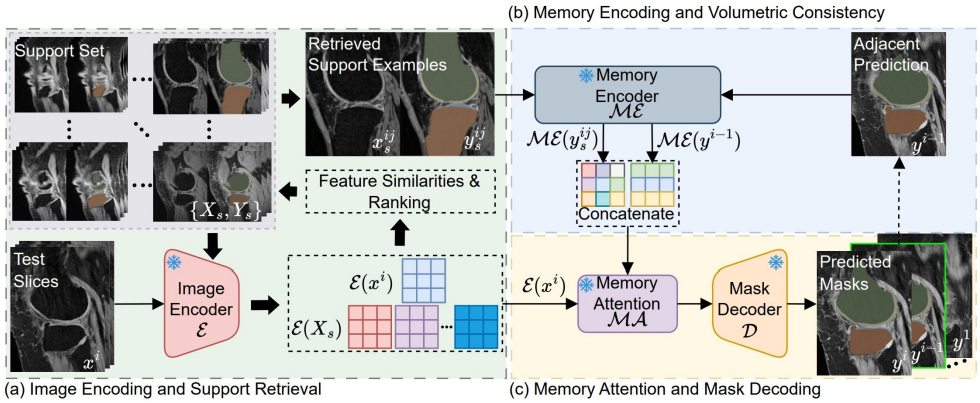


Figure 3.7: Architecture of FATESAM, from He et al. (2025b).

The implementation of FATESAM in the evaluation was based on the official repository (He et al. 2025a), and the checkpoint used was `sam2_hiera_large`, as it was the best performing one comptatible with the repository. Three support images were injected per query image, as this was the number of support images that was the number found to be most optimal in the paper (He et al. 2025b). To adapt FATESAM to single frame images, the object pointer functionality was turned off through Hydra/YAML overrides, which was necessary to avoid shape mismatches. In particular, the object-pointer encoder and object-score prediction heads were disabled by setting `use_obj_ptrs_in_encoder`,

`pred_obj_scores`, `pred_obj_scores_mlp`, and `fixed_no_obj_ptr` all to false. In an ablation study of object pointers in SAM2 (see Ravi et al. 2024, p. 15), its importance was only evaluated on video datasets and included on that basis. It was therefore deemed reasonable to assume that it would not contribute to the performance of FATESAM on single frame images, and thus could be turned off without compromising performance.

3.1.3.4 GFSAM

The other kind of training-free ICL adaptation of SAM tested was the one-shot GFSAM (see A. Zhang et al. 2024). It works by first extracting the features of the given support image with DINOv2 (Oquab et al. 2024), (see Figure 3.8). With its Positive-Negative Alignment module, it then computes two similarity maps between query and support features, one for positive/object similarity, and one for negative/background similarity. These maps are then used to generate foreground point prompts for SAM, which are then fed to the mask decoder that generates one mask per point prompt. A graph is then constructed of the coverage of the points laying within each of the generated masks, whereafter the removal of some of the points representing weakly connected components is determined through processes referred to as Positive Gating and Overshooting Gating. The purpose of those gating processes is to filter out false-positive masks, where the former process makes sure a given point is actually foreground according to the similarity maps, and the latter ensures that the point is not a boundary-near outlier. At the very last, the masks associated with the remaining points are merged together to generate the final segmentation mask.

As an adaptation to the data, the selection of support images for GFSAM was done in the same way as for FATESAM, by ranking the feature similarity of the support images to the input image via Manhattan distance on the extracted features from DINOv2, and selecting the most similar one as one-shot support.

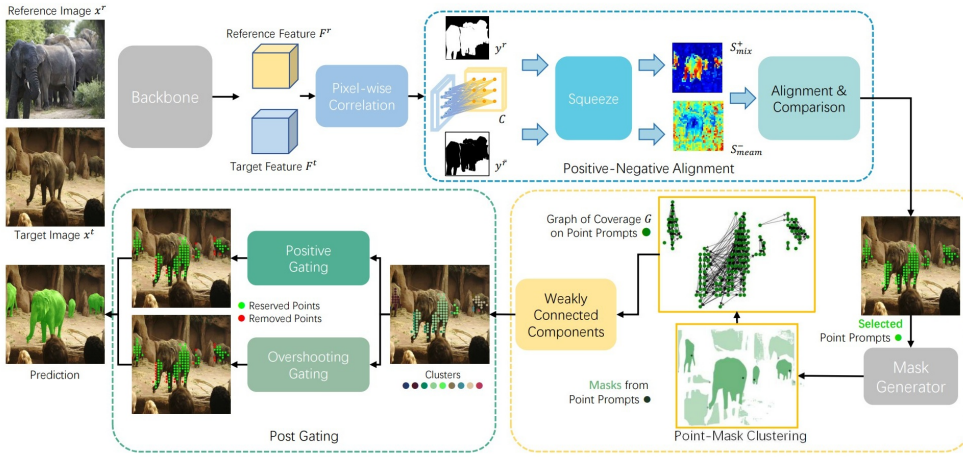


Figure 3.8: Architecture of GFSAM, from A. Zhang et al. (2024).

3.1.3.5 Threshold-inferred automatic point prompts

Another way to adapt SAM to the specific characteristics of the domain was through the use of automatic prompts. As it is particularly difficult to automatically create meaningful box prompts without a trained object detection model, only point prompts were used for this adaptation.

Inspired by Point of Interest (POI) placement in (Price, Rundensteiner, and Cote 2025), automatic point prompts for SAM (see Figure 3.9 for a visual example) were derived from the output mask of the classical Gaussian adaptive thresholding baseline (see Section 3.1.2.3), This was done by randomly sampling a number of foreground and background pixels from the classical output mask as prompts for SAM+pts. Concretely, the foreground cloud of prompts were sampled from the foreground of the mask by n_{fgd} points, while the background cloud of prompts were sampled from the erosion of the inverse of the mask by n_{bgd} points. This gave in total 7 hyperparameters (see Figure A.3) for SAM+pts.

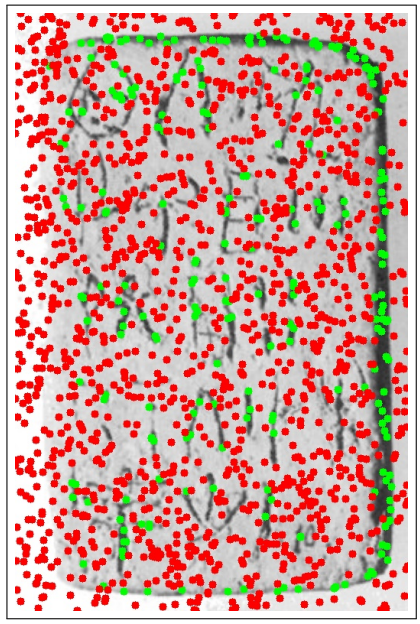


Figure 3.9: A visual example of prompt distribution for SAM with automatic point prompts (SAM+pts), overlayed on HT7a. Green points represent foreground point prompts, while red points represent background point prompts.

3.1.4 Evaluation protocol and performance metrics

For an evaluation of initial segmentation quality, relevant metrics and statistical tests needed to be defined to determine whether one model performed significantly better than another. Furthermore, all models with hyperparameters needed to be optimized to their best performance on the dataset, to ensure a fair comparison. Naturally, as a last check of SAM's efficiency, it was of particular interest to compare its best performing version against the best performing classical baseline.

What follows is a description of the choice of metrics, the optimization of model hyperparameters, and the statistical tests used in the evaluation.

3.1.4.1 Choice of metrics

When it comes to evaluation of segmentation models, the two most commonly used metrics are the Dice-Sørensen coefficient (Dice) and the Intersection over Union (IoU). The Dice score is most commonly used in cases where there are heavy class imbalances (e.g. lots of background compared to regions segmented), and when foreground objects are small and thin. It is naturally more foregiving than the related segmentation metric IoU, which instead tends to be rather strict, and is specifically used to evaluate segmentation results where small displacements are critical for results. Therefore, segmentation performance was reported using Dice. Mathematically, it is defined as twice the size of the intersection of the predicted segmentation mask $\hat{Y}$ and the ground truth mask Y, divided by their summed size:[15]

$$\text{Dice}(\hat{Y}, Y) = \frac{2 \cdot |\hat{Y} \cap Y|}{|\hat{Y}| + |Y|}$$

Thus, the raw metrics ended up consisting of the Dice score for each image, and each model.

3.1.4.2 Hyperparameter optimization

To ensure a fair comparison between the models, all models with hyperparameters were tuned to their best performance on the dataset using Optuna, a hyperparameter optimization framework. The optimization itself was performed using the Tree-structured Parzen Estimator (TPE) algorithm (see Bergstra, Yamins, and Cox 2013)[16], which is a Bayesian optimization method that models the relationship between hyperparameters and performance to efficiently explore the hyperparameter space and find the optimal ones. Hence, it is more sophisticated than a random search, which just samples different hyperparameter combinations without considering the results of past evaluations.

It works by building two probabilistic models: one for the hyperparameter configurations that have resulted in good performance, and

[15] Mohammadi, Mollazade, and Behroozi-Khazaei 2025.

[16] also see Bergstra, Bardenet, and Kégl December 2011.

another for those that have resulted in poor performance. Then, it uses these models to select the next hyperparameter configuration to evaluate, by maximizing the expected improvement over the current best performance. This way, it can focus the search on promising regions of the space of possible hyperparameters without having to rely on chance or exhaustive search.

3.1.4.3 Statistical tests

To determine whether the observed differences in scores between models were statistically significant, paired t-tests were conducted. The paired t-test has two main assumptions: 1. independence of observations, and 2. normality of residuals. The first assumption is already satisfied a priori, since the segmentation performance of one image does not directly influence the performance on another. However, the second assumption depends on results. Thus, the residuals of the paired differences in Dice scores between the two models in question needed to be checked for normality using a QQ-plot and the Shapiro-Wilk test. And in case the normality assumption was violated, the non-parametric domain-specific Wilcoxon signed-rank test would be used instead.[17]

3.2 Interaction efficiency

To answer the question of RQ2: *To which extent can user interaction be reduced by a SAM-based semi-automatic segmentation workflow, while still maintaining high-quality outputs?*, a two-in-one interactive segmentation tool was developed including a method meant to represent SAM, and another to represent the classical segmentation methods, to allow for a direct comparison in terms of tool efficiency. The following subsections go into details on how this tool was developed, the experimental design for the evaluation of interaction efficiency, and the definition of interaction metrics.

[17] Rainio, Teuho, and Klén 2024.

3.2.1 Tool development

The tool was developed in close collaboration with Ester Salgarella, in an attempt to make it fit her workflow and needs as much as possible. However, before any user tests were conducted, a Minimum Viable Product (MVP) was developed based on the workflow that she had described throughout the fortnightly supervisor meetings attended during the writing of this Thesis from February 2nd 2026, and the few sporadic meetings and written exchanges since November 2025 that took place during the planning of the project proposal. When this MVP was ready, three usability tests were conducted with Ester, two of them online, and the last one in-person at Aarhus Institute of Advanced Studies (AIAS).

During these usability tests, Ester was asked to perform the segmentation of at least one document with each method of the tool, while thinking aloud and giving feedback on the user experience. Small modifications were then made to the tool based on her feedback, until the final acceptance test was conducted. These tests took place the 5th, 8th and 12th of May (see Section A.2.1 for documentation).

3.2.2 Tool architecture

The architecture of the tool was developed as a web-application using Svelte as the frontend framework, and FastAPI as the backend framework, hosted via Render. Because it was made a web application, it could be run remotely on any device through a web browser, without the need for installation, making it more easily accessible and ideal for online user testing with Ester Salgarella. To make the development environment more consistent with production, the application was containerized using Docker, which made it easier to deploy and run on any machine without worrying about dependencies and environment setup.

Svelte was chosen for the frontend mostly due to its reactive state management, making it well-suited for building interactive user interfaces, as this functionality removes the need for re-inserting state variables every time they change. FASTAPI was chosen as the backend, as it is Python based, and therefore able to serve the Python segmentation models implemented, which was essential for the tool's functionality. It is also lightweight and asynchronous, which potentially allowed for many

users to use the tool simultaneously, as several requests could be handled at the same time without blocking execution on the server.

As previously mentioned, the tool was split into two segmentation methods: one method meant to represent SAM, and another meant to represent the classical segmentation methods. Besides the general user interface, the following subsections go into more details on how these were implemented.

3.2.2.1 General user interface

The user interface of the tool was designed to be as simple and intuitive as possible, while providing the necessary functionality for the user to perform the segmentation task. The main view of the tool consisted of a canvas where the user could see the image imported (through paste from the clipboard, file upload or drag-and-drop). Once an image was imported, it was displayed on the canvas, and the user could press the "Run" button to generate a mask with the chosen segmentation method on the image. Thus, the user could then toggle between two views: one to see the original image imported; "Raw", and another to see the segmented mask generated therefrom; "Mask" (see Figure 3.10 for an example of this view, using the classical method). And at last, the user could then export this mask in a format of their choice (see Section 3.3.2 for details on this).

3.2.2.2 Integration of the classical method

Chosen to represent classical segmentation methods was the best classical method from the non-interactive evaluation described in the previous section: Gaussian adaptive thresholding (see Section 3.1). On the basis of user feedback during the usability tests (see Section A.2.1), a slider was added as an interactive way to control the coarseness of the segmentation. A range of "granularity" was given $[-50, 100]$, where 0 represented the default hyperparameters of the model. The higher the value of this "granularity" slider, the more coarse the segmented mask would turn out, and the lower the value, the more fine-grained it would be. In that way, the user could adjust the coarseness of the output mask without

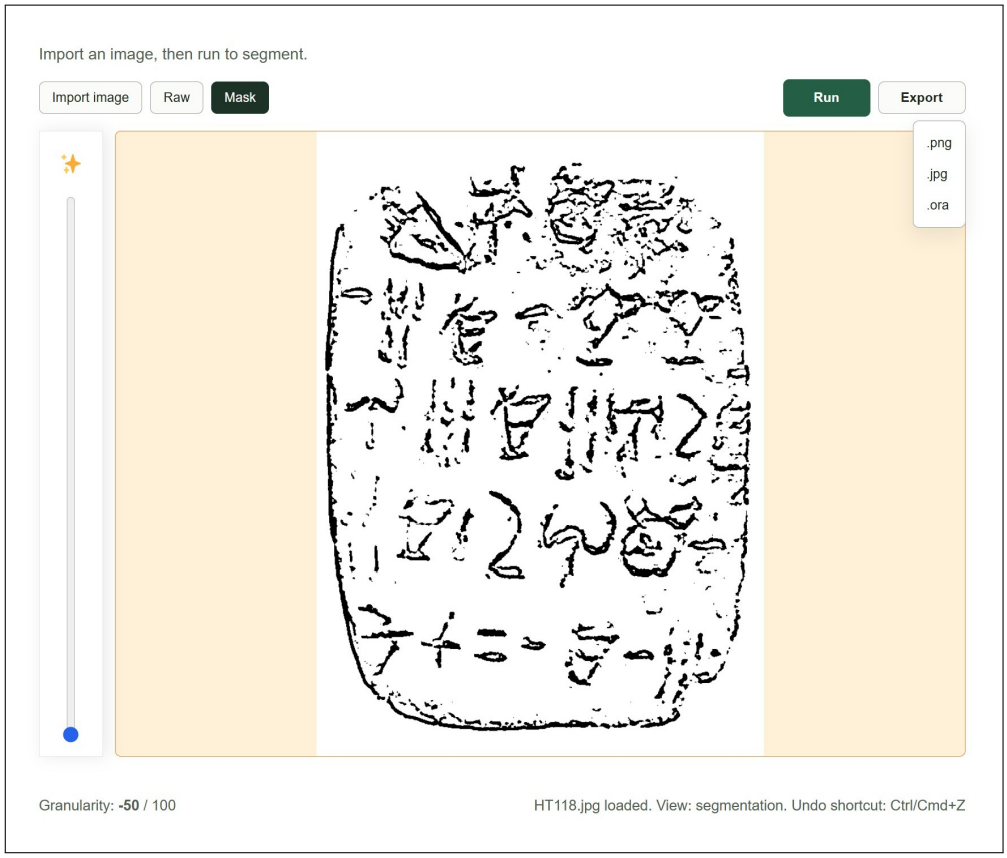


Figure 3.10: In-tool view of the classical Gaussian adaptive thresholding tool in the mask view.

having to worry about the specific hyperparameters of the model, and without having to understand how they work. This worked by defining the Gaussian kernel size D_{Gaussian} as a function of the granularity value as follows (where g is the value of granularity, and $D_{\text{Gaussian,default}}$ denotes the default kernel size of the model):

$$D_{\text{Gaussian}} = \max\left(5,\ D_{\text{Gaussian,default}} + 2\left\lfloor\frac{g}{5}\right\rfloor\right)$$

The granularity value was thus divided by 5 (making the intuitive larger percentage-style range $[-50, 100]$ possible), and then rounded down to the nearest integer, to determine how many times the kernel size

should be increased by 2 (to keep it odd), with a minimum kernel size of 5 to avoid meaningless too fine-grained outputs. A view of this method and the granularity slider can be seen in Figure 3.10, where the slider is located in the toolbar on the left.

3.2.2.3 Integration of SAM

Chosen to represent SAM was the vanilla version of SAM2. This choice was made due to its theoretical SOTA superiority, as it was deemed that the non-interactive evaluation described in the previous section could not be used to conclude how the methods would perform when used interactively. It was decided to make full use of the interactive capabilities, including both box prompts, and foreground and background point prompts, as this seemed to be the best way to represent its potential. But there was one problem. The mask decoder for SAM (and SAM2 for that matter) only takes one box prompt at a time, which is not ideal when wanting to segment multiple signs of interest within the same tablet. To circumvent this, the prompts given to it when boxes were present were processed in the mask decoder so that a separate mask was generated once for each n box prompts placed, also passing the point prompts within it (see Figure 3.11 for the associated user interface). These n masks were then at last merged together by a pixel-wise logical OR operation to generate the final segmentation mask. This was still sensible computationally, as predictions were split up such that the image encoder was only executed once on image import, while the mask decoder (as described in Section 3.1.3.1) is lightweight enough that it could be executed multiple times without a significant increase in latency.

Included in this prompt placement interface of the tool was a toolbar consisting of the following tool buttons (see Figure 3.11 and Figure 3.12 for in-tool views of the SAM-based tool, with the toolbar on the left):

- `Box prompt`: When selected, allowed the user to place box prompts on the canvas by clicking the pixelcoordinates of the corners that should define the box.

- `Foreground prompt`: When selected, allowed the user to place foreground point prompts on the canvas by clicking on the pixel coor-

dinates they want to include in the segmentation.

- `Background prompt`: When selected, allowed the user to place background point prompts on the canvas by clicking on the pixel coordinates they want to exclude from the segmentation.
- `Eraser`: When selected, allowed the user to remove any placed prompts by clicking on them.
- `Undo`: When selected, allowed the user to undo the last prompt-related action performed.
- `Reset`: When clicked, cleared all prompts from the canvas, allowing the user to start over.

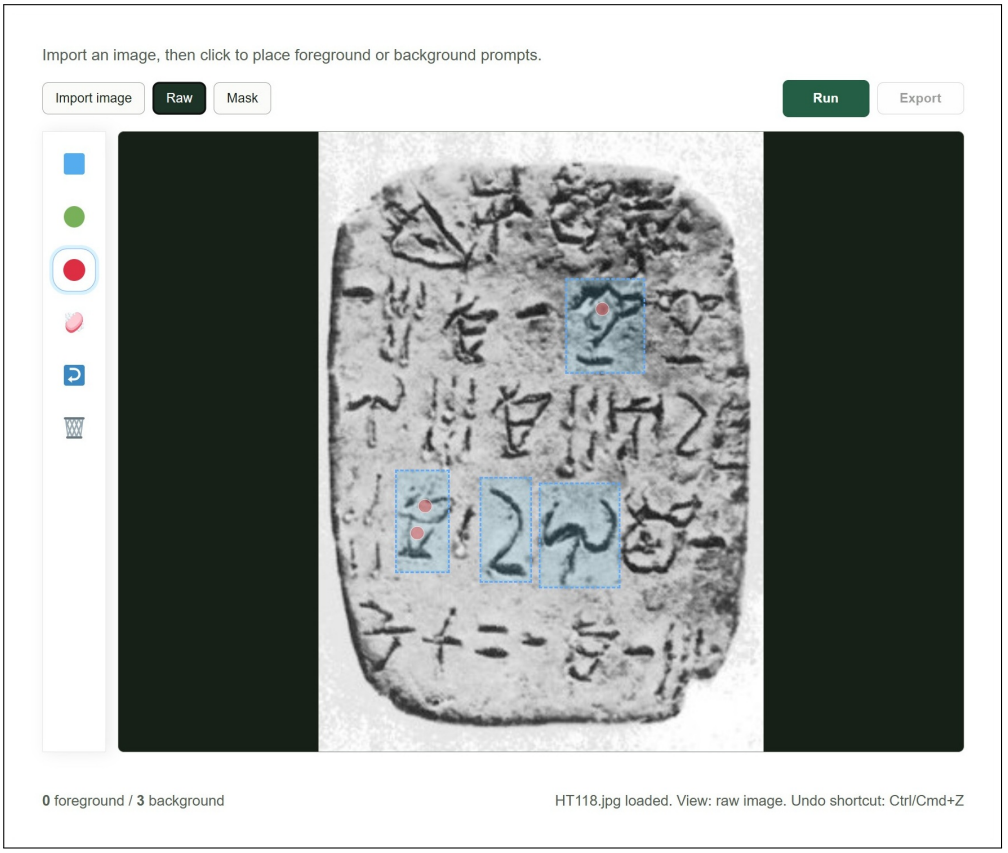


Figure 3.11: In-tool view of the SAM-based tool in the raw view, with box prompts (in blue) placed around some signs of interest, with correcting background point prompts (in red) placed within the oversegmented regions.

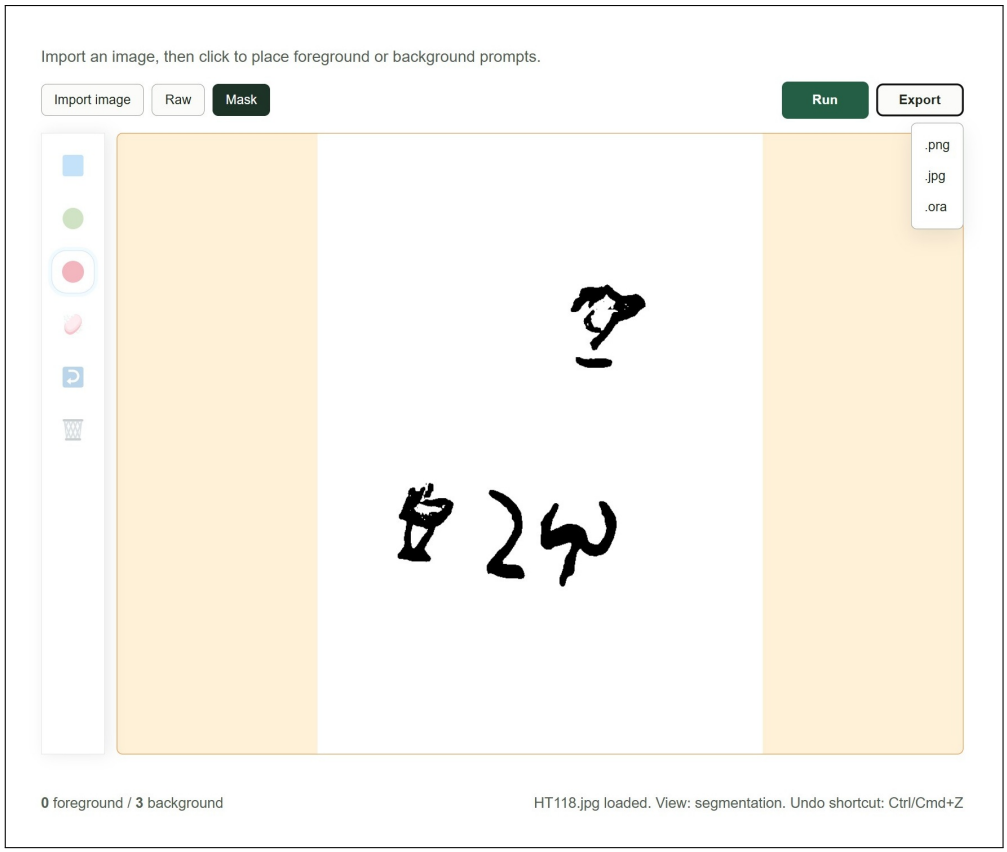


Figure 3.12: In-tool view of the SAM-based tool in the mask view, before export of the mask generated from the prompts placed as seen in Figure 3.11.

3.2.3 Experimental design for evaluation of interaction efficiency

To evaluate the interaction efficiency of the SAM-based tool, a case study was conducted with Ester Salgarella, where she was asked to perform the segmentation of 3 documents from the test set (one from each difficulty category) with each method of the tool. In a wish to obtain statistical power, an inclusion of all 27 test images was initially preferred, but due to time constraints, only 3 documents made it to the evaluation. That reduced sample size made the evaluation exploratory and case-study-based rather than statistically powered. However, it still allowed for a direct comparison of the interaction efficiency of the two tools, using quantitative indicators of both time expenditure (minutes), and the quality of the outputs they generated (Dice). The interaction burden could then further be exploratorily decomposed into the time taken for the models to execute, and for SAM-based tool, the amount and types of prompts placed.

The experiment itself was conducted in-person with Ester Salgarella as the participant at Aarhus Institute of Advanced Studies (AIAS) on the 12th of June on the same day as the last usability and acceptance test. Specifically, she was instructed to make use of the tool method evaluated on the document given until she deemed that she could not further improve the segmentation output, whereafter she should export the final mask as it was. Tool interaction was recorded by the help of live screen recordings via Microsoft Teams, wherefrom the model execution time, the prompts placed, and the time taken from image import to mask export were all measured. The exports of each tool were also saved on disk, so that Dice scores could later be computed therefrom.

3.3 Workflow efficiency

To answer the question of RQ3: *How much reduction in annotation time can be achieved using the proposed tool compared to fully manual digitization, under controlled experimental conditions?*, another case study of 3 documents was conducted with Ester Salgarella, where the time

taken to finish polishing the segmentation outputs exported from the tools described in the previous section (see Section 3.2) was measured, and compared to the time taken to do the same drawings fully manually.

The following subsections go into more details on the precautions taken in an attempt to fit the tool into the workflow, the experimental design for the evaluation of workflow efficiency, and the definition of the workflow efficiency metric.

3.3.1 Definition of the workflow

The tool was designed to fit into Ester's existing workflow as much as possible, to make it more likely that she would be able to use it with minimal friction, and to make the evaluation of its efficiency more ecologically valid. To do that, however, it was necessary to first understand her workflow in detail.

When Ester does the annotation of a document from the GORILA corpus, she first imports a screenshot of the GORILA corpus drawing into Krita, creating a new layer on top of it to do the segmentation drawing. She draws this drawing with a digital pen on a displayless pressure-sensitive Wacom drawing pad. And while doing that, she looks for any possible mistakes in the signs, with the original image, and her mental map of the sign types as guidance, correcting them by erasing and redrawing. At last, when she is satisfied with the drawing, she splits each sign into a separate layer, adds metadata, and exports to JSON format. Due to scope limitations, it was only the process before that splitting into layers and adding of metadata that this project aimed to improve.

3.3.2 Export to Krita

When the segmentation of a document within the tool was complete, it was designed to allow for export to JPEG, PNG, or OpenRaster (`.ora`) format. The latter is an open file format that like Krita (`.kra`) files consists of a ZIP file with an XML document describing the layer structure, together with a PNG or SVG image for each raster or vector layer, respectively. This is simpler in structure and easier to read and write for

external programs than the `.kra` file format, since it was specificially designed as an open exchange format between graphics editors. Krita files, on the other hand, are more complex in structure, including proprietary elements, and not designed for external programmatic access, as they are meant to be used within Krita only.

Additionally, OpenRaster (`.ora`) format is supported by a few image editing programs, including Krita, which makes it ideal for the exported file to be directly opened there for further manual editing and annotation. And as Krita was already part of Ester's workflow, it was only natural to make the exported starting point suggestions from the tools directly compatible with it, to make the transition from tool to manual post-editing as seamless as possible. This file format also made it possible for the tool export to include the original image as a layer, and the generated mask as another, so that the former could be used as a guiding reference for post-editing of the latter.

3.3.3 Experimental design for evaluation of workflow efficiency

The workflow efficiency case study was built upon the data obtained from the in-tool case study described in the previous section (see Section 3.3.1). These two case studies can therefore be seen as a single one, as one was merely an extension of the other. Thus, both of them suffered from the same time-constraint-caused limitations in terms of sample size and statistical power, making them exploratory and case-study-based rather than statistically powered.

The drawings were made in Krita using the displayless pressure-sensitive Wacom drawing pad with a pen input used in her original workflow (see). The case study involved Ester Salgarella post-editing the tool exports from the interaction efficiency study in Krita, recording the time taken, to compare it to the time taken to do the same drawings fully manually. Since Ester expressed a lack of guiding reference to know when the drawing was complete - even with the originally imported raw tablet images included as layers - the corresponding GORILA corpus drawing for each document was also included as a layer in the ex-

ported `.ora` file by manually importing it and aligning it with the other two layers before letting Ester start the post-editing. And as a proof of completion, Ester was asked to send the finished `.ora` files back again.

CHAPTER 4

Results and Discussion

4.1 Non-interactive segmentation performance

After running the non-interactive segmentation evaluation of the models on the the test set (hyperparameter optimization results can be found in Section A.1.1.2), it was clear that SAM did not perform better than the classical baselines (see Figure 4.4 for a plot of the results).

The following subsections go into the small ablation study of the basic SAM variants, the performance of the ICL variants, the comparison to classical baselines, and the statistical significance of the differences in performance between the best of these and SAM (see Section A.1.2 in the appendix for the full results table).

4.1.1 Ablation of zero-shot SAM

The results of the ablation study of different SAM variants (see Figure 4.1) showed that the best performing variant was the original SAM with image encoder Vit-H and automatic point prompts, without any filter.

4.1.2 Comparison of ICL SAM

Neither GFSAM nor FATESAM performed better than the best zero-shot SAM implementation (see Figure 4.2). This is not too surprising given the small size and poor quality of the dataset, and poor alignment of support images with ground truth.

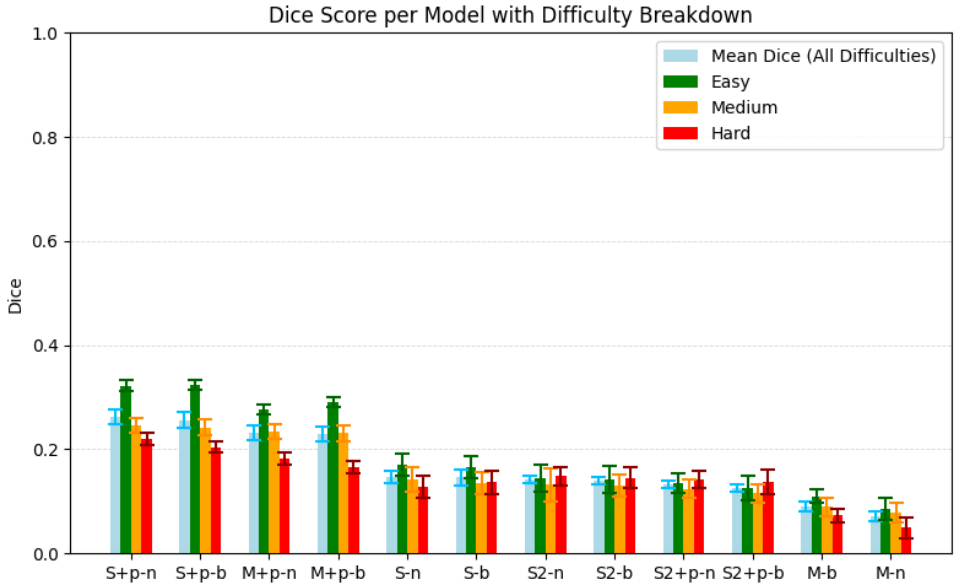


Figure 4.1: Dice Score performance per SAM variant. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance. The ablations evaluated were MobileSAM (M), SAM (S), and SAM2 (S2), with (+p) or without automatic point prompts, and with (b) or without (n) a bilateral pre-filter.

4.1.3 Classical model comparison

With a mean Dice score of around 0.263, the best performing SAM variant performed better than Otsu and CannyFill (with mean Dice scores of 0.224 and 0.183, respectively), but clearly underperformed compared to the best performing classical baseline, Gaussian that yielded a Dice score of 0.327 (see **??** for details). See Figure 4.5 for a visual comparison of the output masks of the best SAM variant and the classical baselines on one of the easy images, and Section A.1.2 for the full results table.

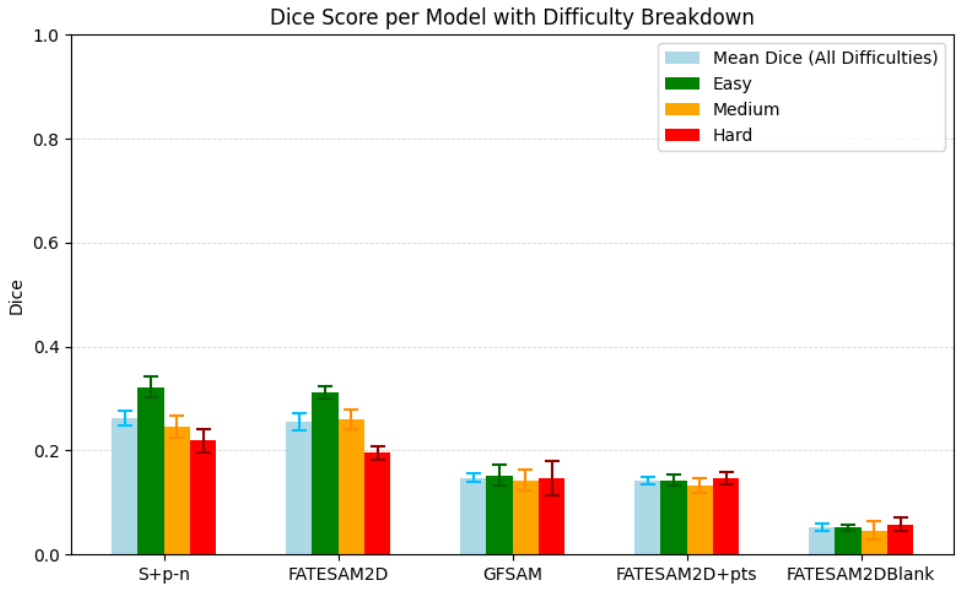
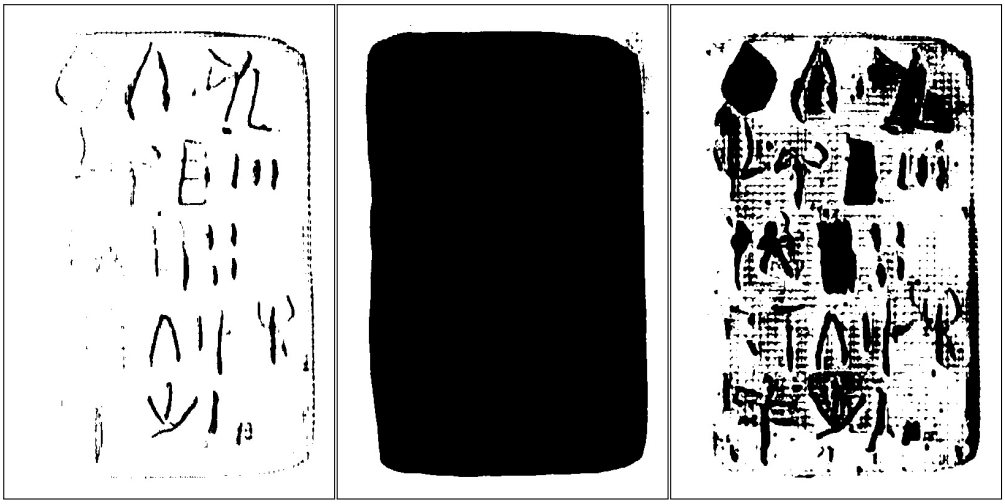


Figure 4.2: Dice score comparison of the ICL models against the best performing zero-shot SAM implementation. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance. Compared were GFSAM, the 2D-adapted FATESAM2D, its automatic point prompted version FATESAM2D+pts, and FATESAM2DBlank, which was given blank support images.

((a)) Raw Image ((b)) Ground Truth



((c)) FATESAM2D 0.37 ((d)) GFSAM 0.13 ((e)) S+p-n 0.26

Figure 4.3: Visual comparison of output masks per ICL model and best SAM variant from Section 4.1.1 of the segmentation on PH2 (easy), shown with Dice scores next to the raw image and ground truth. This is one of the instances where FATESAM2D performed better than the best zero-shot SAM implementation, but due to general undersegmentation, this was generally not the case.

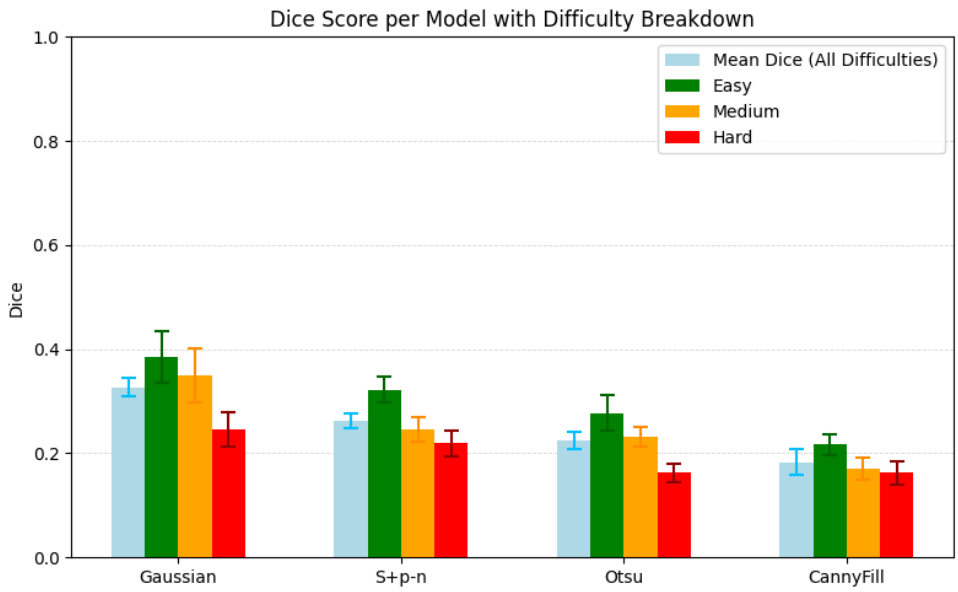
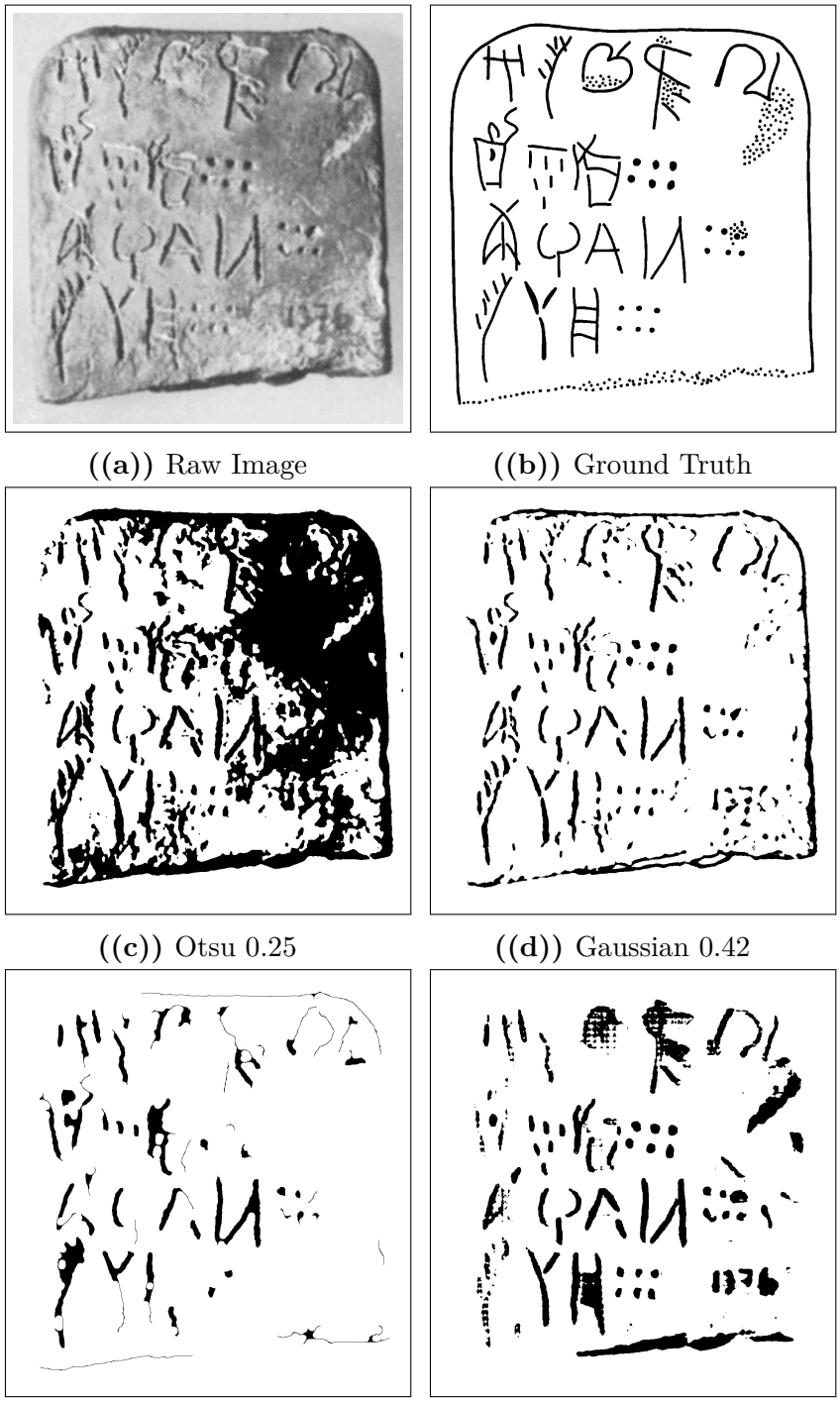


Figure 4.4: Dice score comparison of all models on the evaluation dataset. Split by difficulty. With error bars representing the standard error of the mean. Ordered descendingly from the left by mean performance.



((**a**)) Raw Image ((**b**)) Ground Truth

((**c**)) Otsu 0.25 ((**d**)) Gaussian 0.42

((**e**)) CannyFill 0.32 ((**f**)) S+p-n 0.38

Figure 4.5: Visual comparison of output masks per classical model and best SAM variant of segmentation on PH2 (easy), shown with Dice scores next to the raw image and ground truth.

4.1.4 Statistical significance

As it was clear from the results, SAM+pts did not perform better than the best performing classical baseline. To determine whether it performed significantly worse, a paired t-test was conducted, as residuals of the differences in Dice scores between Gaussian and SAM+pts were approximately normally distributed (see next Section 4.1.4.1). What follows is the documentation of that normality test, the paired t-test, and the results of both.

4.1.4.1 Normality of residuals

As the QQplot in Figure 4.6 shows, the residuals of the paired differences in Dice scores between Gaussian and SAM+pts appeared to be approximately normally distributed. Furthermore, the Shapiro-Wilk test for normality of these residuals yielded a p-value of $0.056 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected. Thus, the normality assumption for the paired t-test was satisfied, and its results could be considered valid.

4.1.4.2 Paired t-test

The paired t-test was conducted on the Dice scores obtained for each of the 27 images across Gaussian and SAM+pts. The null hypothesis (H_0) stated that there is no difference in mean Dice scores between the two models, while the alternative hypothesis (H_1) stated that Gaussian has a higher mean Dice score than SAM+pts.

Performing the paired t-test yielded a test statistic of $t \approx 3.737$ and a p-value of $P(T > t) \approx 0.001$. Thus, as $0.001 < \alpha = 0.05$, the null hypothesis could be rejected, and it could be concluded that Gaussian significantly outperforms SAM+pts in terms of Dice score. Hence, it can be said that SAM performed significantly worse in comparison to the best classical segmentation methods for non-interactive segmentation of Linear A tablets, even when automatically prompted.

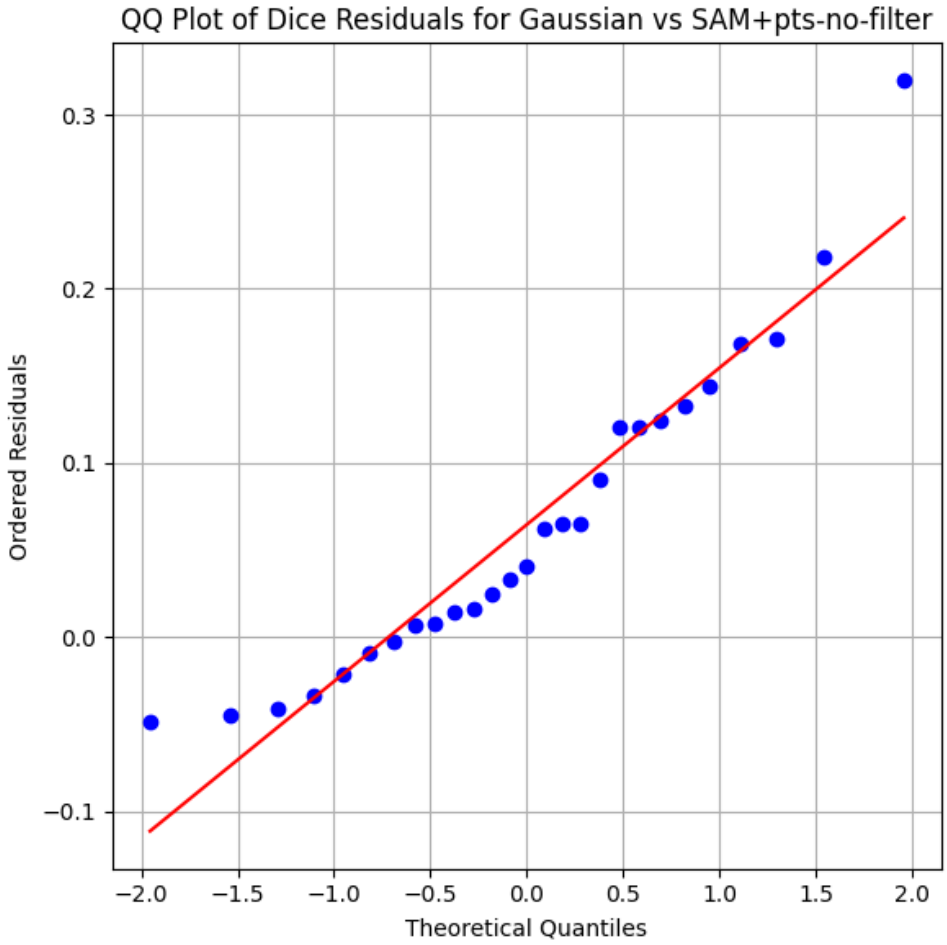


Figure 4.6: QQplot of the residuals of the paired t-test comparing Gaussian and SAM+pts. The residuals appear to be approximately normally distributed, which supports the validity of the paired t-test results. Furthermore, the Shapiro-Wilk test for normality of these residuals yielded a p-value of $0.056 > \alpha = 0.05$, indicating that the null hypothesis of normality could not be rejected.

4.1.5 Fairness of the experiment

4.1.5.1 Model implementation

In the case of the fairness of the SAM implementation chosen, the experiment obviously only evaluates the performance of an untuned version. This implementation had most certainly never seen Linear A tablet images during training, as it was pretrained by default on the large and diverse dataset of natural images SA-1B. Furthermore, SAM was restricted by the automatic nature of this experiment, as its intended use is interactive segmentation. Thus, it does not come as a surprise that SAM underperforms in comparison to the classical baselines. The results therefore suggest a need for human intervention for the optimal placement of prompts to provide the best conditions for SAM, as it otherwise underperforms.

4.2 Tool-side segmentation performance

The case study of 3 documents showed that the interactive version of SAM2 with box prompts and point prompts not only took more time to use than the classical baseline, due to the burden of interaction, but also did not perform better in terms of segmentation quality, even with the help of human input (see Figure 4.7).

A significant portion of the time spent using the interactive version of SAM2 was on waiting for the model to execute through Modal (see Figure 4.8). But clearly this was not the bottleneck, as the time spent on interaction was much higher, taking up more than 80% of tool time. The time spent waiting on SAM execution was also below 5 seconds more than 50% of executions.

Looking at the interaction patterns of the SAM-based tool, it is clear that the amount of prompts placed per image was quite high, with all instances above 60 prompts per image (see Figure 4.9). Only background point prompts were placed, suggesting that the user was compensating only for over-segmentation.

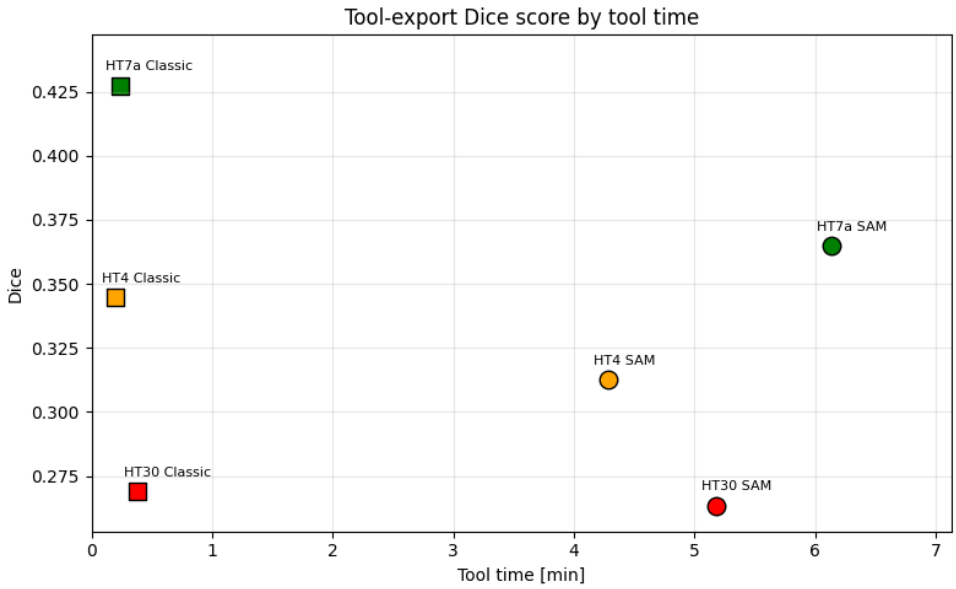


Figure 4.7: Dice scores of the exported masks as a function of tool time, comparing the SAM-based tool to the tool based on the classical baseline.

4.3 Workflow impact

The case study of 3 documents showed that none of the two tools were able to save time in the workflow, as the total workflow time spent was higher than the time spent on manual annotation in Krita (see Figure 4.11). Even just based on the time spent on post-editing alone, the tools did not save time compared to just drawing manually from scratch. In fact, the total time spent doing the classical workflow was more than twice the time spent doing the manual one, while the time spent doing the SAM-based workflow was more than thrice (and almost four times) the time spent on the manual one.

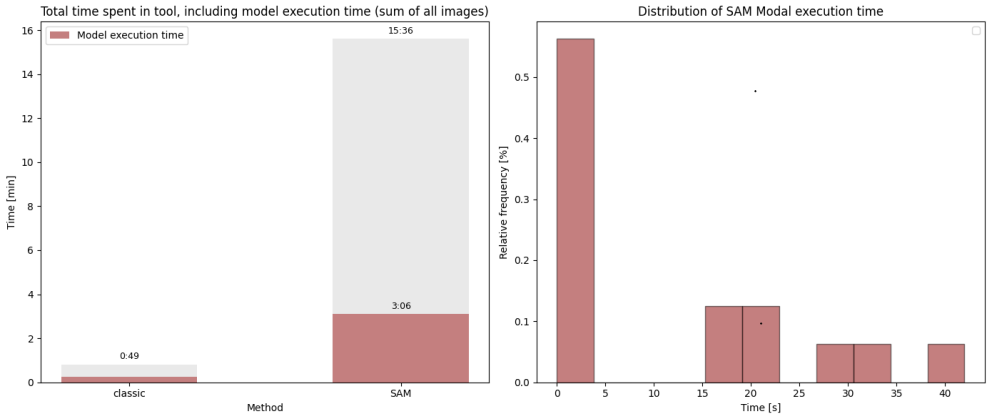


Figure 4.8: Tool time spent on model execution per method (left), and distribution of SAM Modal interaction time (right).

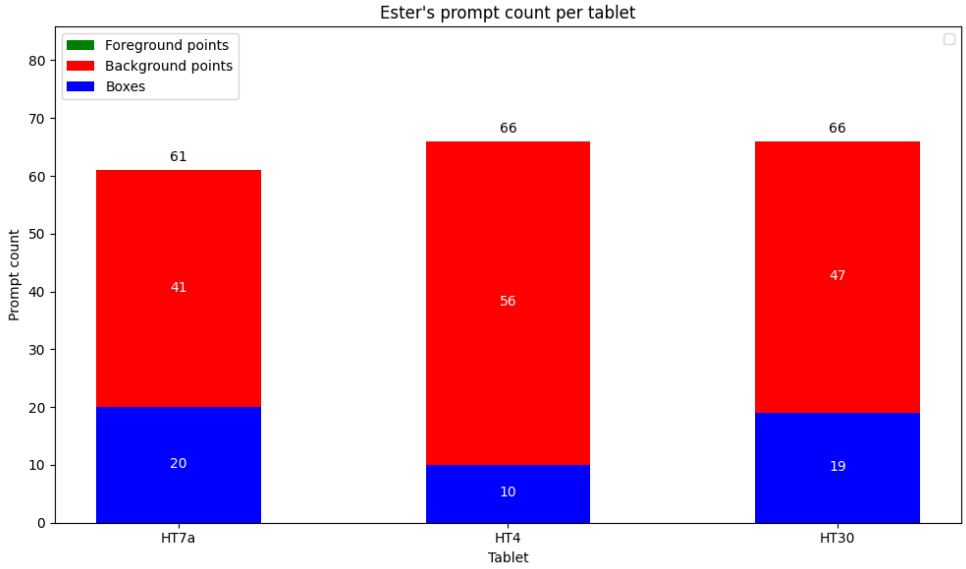


Figure 4.9: The amount of prompts placed per image using the interactive SAM-based tool.

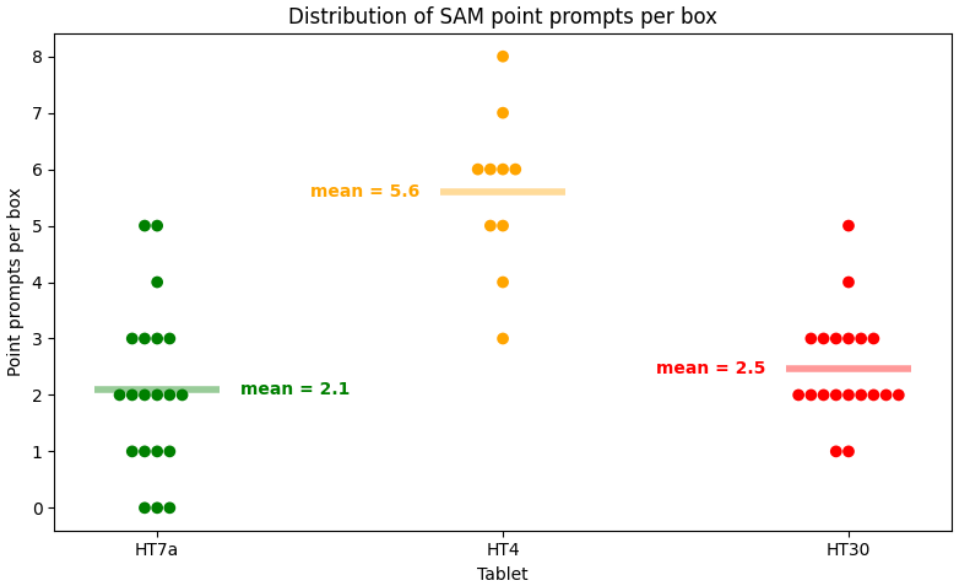


Figure 4.10: Average number of point prompts placed per box prompt for the interactive version of SAM2.

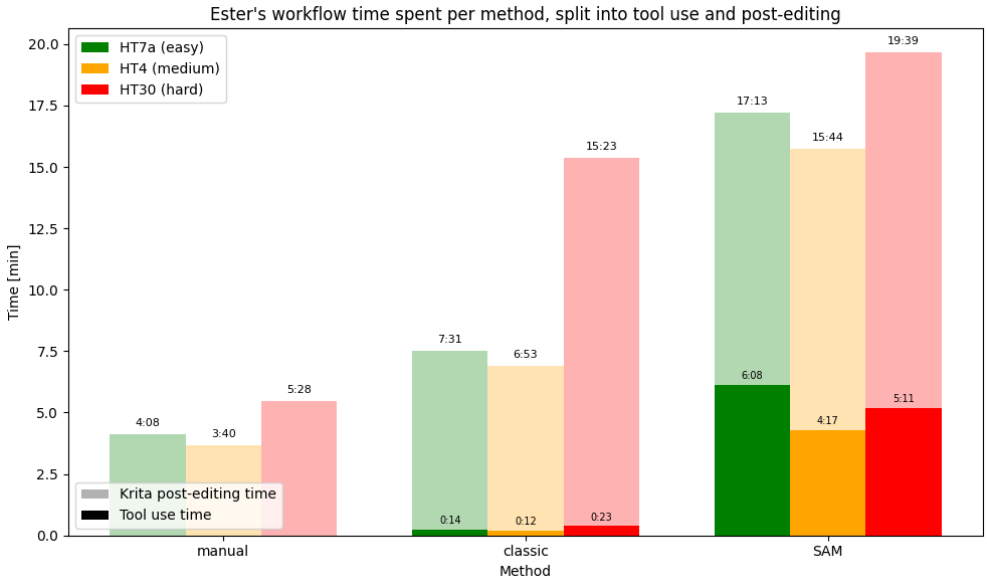


Figure 4.11: Workflow time spent per method, split into phases of tool use (opaque colors) and post-editing in Krita (semi-transparent colors).

CHAPTER 5

Conclusion

5.1 Summary of findings per research question

5.1.1 RQ1: Segmentation performance

5.1.2 RQ2: Interaction efficiency

5.1.3 RQ3: Workflow efficiency

5.2 Overall contribution

5.3 Future Work

5.3.1 Alternative data sources

Since the raw tablet based ML system developed did not perform well enough to be deemed helpful for the workflow, another approach seems necessary in order to achieve a more automated workflow. The bottleneck of the raw tablet based approach seems to be the quality of the input data, as the raw images are of low quality and contain a lot of noise, which is not only hard for the model to process, but also even for the researcher to differentiate the signs from mere tablet damage. The suggestions generated should therefore instead be inferred from the corpus drawings, which are of higher quality, carry more strucured information than the

raw tablet images, and are what the researcher already primarily uses as reference in the annotation process.

5.3.2 Methodological extensions

5.3.3 Further automation opportunities

...The results therefore suggest a need for a generalized and computationally efficient SAM implementation to be tuned to the greater domain of inscriptions, as the default model is not sufficient enough to prove its potential capabilities there. For this potential tuning to be possible, there is a need of a sufficiently sized one-class-labeled dataset of general inscription segmentations.

Bibliography

Bergstra, James, Remi Bardenet, and Balázs Kégl (December 2011). "Algorithms for Hyper-Parameter Optimization." In.

Bergstra, James, Daniel Yamins, and David D. Cox (2013). "Making a Science of Model Search: Hyperparameter Optimization in Hundreds of Dimensions for Vision Architectures." In: *Proceedings of the 30th International Conference on Machine Learning (ICML)*. Volume 28. Proceedings of Machine Learning Research 1. PMLR, pages 115–123. URL: https://proceedings.mlr.press/v28/bergstra13.pdf.

Carion, Nicolas et al. (2025). *SAM 3: Segment Anything with Concepts*. arXiv: 2511.16719 [cs.CV]. URL: https://arxiv.org/abs/2511.16719.

Christoffersen, Frederik W. L. (2026). *Source code for this Thesis*. Source code repository. URL: https://github.com/FrederikWLC/linA-scribe.

Consiglio Nazionale delle Ricerche (2025). *A Major Addition to the Linear A Corpus*. URL: https://www.cnr.it/en/news/13529/a-major-addition-to-the-linear-a-corpus (visited on February 4, 2026).

Del Freo, Maurizio and Julien Zurbach (2025). *Recueil des inscriptions en linéaire A. Supplément 1 (RILA-S1)*. Athens: École française d'Athènes.

Dosovitskiy, Alexey et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv: 2010.11929 [cs.CV]. URL: https://arxiv.org/abs/2010.11929.

Godart, Louis and Jean-Pierre Olivier, editors (1976-1985). *Études crétoises. Recueil des inscriptions en linéaire A (GORILA)*. Five-volume corpus of Linear A inscriptions (GORILA I–V). Athènes. URL: https://cefael.efa.gr/result.php?site_id=1&serie_id=EtCret.

Gonzalez, Rafael C. and Richard E. Woods (2018). *Digital Image Processing*. 4th edition. New York: Pearson. ISBN: 978-0133356724. URL:

https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf.

Hamplová, Adéla et al. (2024). "Instance Segmentation of Characters Recognized in Palmyrene Aramaic Inscriptions." In: *Computer Modeling in Engineering & Sciences* 140.3, pages 2869–2889. ISSN: 1526-1506. DOI: 10.32604/cmes.2024.050791. URL: http://www.techscience.com/CMES/v140n3/57249.

Han, Dongsheng et al. (2023). *Segment Anything Model (SAM) Meets Glass: Mirror and Transparent Objects Cannot Be Easily Detected*. arXiv: 2305.00278 [cs.CV]. URL: https://arxiv.org/abs/2305.00278.

He, Xingxin et al. (2025a). *FATE-SAM: Few-Shot Adaptation of Training-Free Foundation Model for 3D Medical Image Segmentation*. GitHub repository. URL: https://github.com/I3Tlab/FATE-SAM (visited on May 23, 2026).

— (2025b). *Few-Shot Adaptation of Training-Free Foundation Model for 3D Medical Image Segmentation*. arXiv: 2501.09138 [cs.CV]. URL: https://arxiv.org/abs/2501.09138.

Kirillov, Alexander et al. (2023). "Segment Anything." In: *arXiv preprint arXiv:2304.02643*. URL: https://arxiv.org/pdf/2304.02643.

Ma, Jun et al. (January 2024). "Segment anything in medical images." In: *Nature Communications* 15.1. ISSN: 2041-1723. DOI: 10.1038/s41467-024-44824-z. URL: http://dx.doi.org/10.1038/s41467-024-44824-z.

Meta AI Research (2024). *Segment Anything Model 2*. GitHub repository. URL: https://github.com/facebookresearch/sam2 (visited on May 23, 2026).

Modal Labs (2026). *Modal Documentation*. Documentation for the Modal cloud execution platform. URL: https://modal.com/docs/guide (visited on May 23, 2026).

Mohammadi, Mobin, Kaveh Mollazade, and Nasser Behroozi-Khazaei (2025). "Under- and over-segmentation: New metrics for image segmentation accuracy measurement." In: *Array* 28, page 100624. ISSN: 2590-0056. DOI: https://doi.org/10.1016/j.array.2025.100624. URL: https://www.sciencedirect.com/science/article/pii/S2590005625002516.

Oliveira, Saïd, Benoît Seguin, and Frédéric Kaplan (2018). "dhSegment: A generic deep-learning approach for document segmentation." In: *arXiv preprint arXiv:1804.10371*. URL: https://arxiv.org/abs/1804.10371.

OpenCV (2024). *cv::AdaptiveThresholdTypes — OpenCV Documentation*. OpenCV. URL: https://docs.opencv.org/4.x/d7/d1b/group__imgproc__misc.html#gaa42a3e6ef26247da787bf34030ed772c (visited on March 7, 2026).

— (2026). *Image Filtering*. URL: https://docs.opencv.org/4.x/d4/d86/group__imgproc__filter.html#ga9d7064d478c95d60003cf839430737ed (visited on March 23, 2026).

Oquab, Maxime et al. (2024). *DINOv2: Learning Robust Visual Features without Supervision*. arXiv: 2304.07193 [cs.CV]. URL: https://arxiv.org/abs/2304.07193.

Otsu, Nobuyuki (1979). "A Threshold Selection Method from Gray-Level Histograms." In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1, pages 62–66. DOI: 10.1109/TSMC.1979.4310076. URL: https://engineering.purdue.edu/kak/computervision/ECE661.08/OTSU_paper.pdf.

Pan, Cong (2025). "Real-Time Hardware Architecture Design for An Innovative Bilateral Filtering Algorithm." In: *Proceedings of the 2025 5th International Conference on Automation Control, Algorithm and Intelligent Bionics (ACAIB '25)*. New York, NY, USA: ACM, pages 102–107. ISBN: 9798400714313. DOI: 10.1145/3760269.3760285. URL: https://dl.acm.org/doi/10.1145/3760269.3760285 (visited on March 23, 2026).

Price, Stephen, Elke Rundensteiner, and Danielle L. Cote (2025). "Mastering SAM Prompts: A Large-Scale Empirical Study in Segmentation Refinement for Scientific Imaging." In: *Transactions on Machine Learning Research*. ISSN: 2835-8856. URL: https://openreview.net/forum?id=cWcTQMpqv6.

Rainio, Oona, Jarmo Teuho, and Riku Klén (2024). "Evaluation metrics and statistical tests for machine learning." In: *Scientific Reports* 14, page 6086. DOI: 10.1038/s41598-024-56706-x. URL: https://doi.org/10.1038/s41598-024-56706-x.

Ravi, Nikhila et al. (2024). *SAM 2: Segment Anything in Images and Videos*. arXiv: 2408.00714 [cs.CV]. URL: https://arxiv.org/abs/2408.00714.

Rosebrock, Adrian (2015). *Zero-parameter, automatic Canny edge detection with Python and OpenCV*. Accessed: 2026-04-06. URL: https://pyimagesearch.com/2015/04/06/zero-parameter-automatic-canny-edge-detection-with-python-and-opencv/.

Ryali, Chaitanya et al. (2023). *Hiera: A Hierarchical Vision Transformer without the Bells-and-Whistles*. arXiv: 2306.00989 [cs.CV]. URL: https://arxiv.org/abs/2306.00989.

Salgarella, Ester (2020). *Aegean Linear Script(s): Rethinking the Relationship Between Linear A and Linear B*. Cambridge: Cambridge University Press.

— (2025). *Writing in Bronze Age Crete: 'Minoan' Linear A*. Cambridge: Cambridge University Press.

Salgarella, Ester and Simon Castellan (2020). *SigLA: The Signs of Linear A: a Palaeographical Database*. Research Paper / Technical Report. Accessed: 2026-02-04. SigLA. URL: https://sigla.phis.me/paper.pdf.

SigLA (2021a). *About SigLA*. URL: https://sigla.phis.me/about.html (visited on February 4, 2026).

— (2021b). *SigLA Document Search Interface*. URL: https://sigla.phis.me/search-document.html (visited on February 4, 2026).

Tang, Lv, Haoke Xiao, and Bo Li (2023). *Can SAM Segment Anything? When SAM Meets Camouflaged Object Detection*. arXiv: 2304.04709 [cs.CV]. URL: https://arxiv.org/abs/2304.04709.

Tomasi, Carlo and Roberto Manduchi (1998). "Bilateral Filtering for Gray and Color Images." In: *Proceedings of the Sixth International Conference on Computer Vision*. Bombay, India: IEEE, pages 839–846. DOI: 10.1109/ICCV.1998.710815. URL: https://users.soe.ucsc.edu/~manduchi/Papers/ICCV98.pdf (visited on March 23, 2026).

Vaswani, Ashish et al. (2023). *Attention Is All You Need*. arXiv: 1706.03762 [cs.CL]. URL: https://arxiv.org/abs/1706.03762.

Wang, Shibin et al. (2025). "OBIFlo-SAM: multi-task semantic recognition and segmentation of oracle bone inscription." In: *npj Heritage*

Science 13, page 588. DOI: 10.1038/s40494-025-02157-0. URL: https://www.nature.com/articles/s40494-025-02157-0.

Wu, Kan et al. (2022). "TinyViT: Fast Pretraining Distillation for Small Vision Transformers." In: *arXiv preprint arXiv:2207.10666*. arXiv: 2207.10666 [cs.CV]. URL: https://arxiv.org/abs/2207.10666.

Ye, Maoyuan et al. (2024). *Hi-SAM: Marrying Segment Anything Model for Hierarchical Text Segmentation*. arXiv: 2401.17904 [cs.CV]. URL: https://arxiv.org/abs/2401.17904.

Zhang, Anqi et al. (2024). *Bridge the Points: Graph-based Few-shot Segment Anything Semantically*. arXiv: 2410.06964 [cs.CV]. URL: https://arxiv.org/abs/2410.06964.

Zhang, Chaoning et al. (2023). "Faster Segment Anything: Towards Lightweight SAM for Mobile Applications." In: *arXiv preprint arXiv:2306.14289*. URL: https://arxiv.org/pdf/2306.14289.

Zhang, Pan, Chao Li, and Yuanhua Sun (2024). "Stone Inscription Image Segmentation Based on Stacked-UNets and GANs." In: *Discover Applied Sciences* 6.10, page 550. DOI: 10.1007/s42452-024-06264-8. URL: https://doi.org/10.1007/s42452-024-06264-8.

Zhao, Xu et al. (2023). "MobileSAMv2: Faster Segment Anything to Everything." In: *arXiv preprint arXiv:2312.09579*. URL: https://arxiv.org/pdf/2312.09579.

APPENDIX A

An Appendix

A.1 Non-interactive segmentation results

A.1.1 Hyperparameters

A.1.1.1 General hyperparameters

X

1. C: the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.1.2.3),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.1.2.3),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral prefiltering (see Section 3.1.2.3),
4. σ: the spatial and intensity standard deviation for bilateral prefiltering (see Section 3.1.2.3),

Figure A.1: Hyperparameters for Gaussian adaptive thresholding (Gaussian).

A.1.1.2 Optimization results

X

1. σ: the constant used to determine the thresholds T_l and T_h for the hysteresis thresholding,
2. $d_{closing}$: the diameter of the kernel window for the morphological closing of the edges

Figure A.2: Hyperparameters for Canny edge detection with morphological closing (CannyFill).

X

1. C: the constant subtracted from the Gaussian-weighted sum to compute the local threshold (see Section 3.1.2.3),
2. $d_{gaussian}$: the diameter of the kernel window for Gaussian adaptive thresholding (see Section 3.1.2.3),
3. $d_{bilateral}$: the diameter of the kernel window for bilateral prefiltering (see Section 3.1.2.3),
4. σ: the spatial and intensity standard deviation for bilateral prefiltering (see Section 3.1.2.3),
5. n_{fgd_points}: the number of foreground point prompts to be sampled from the foreground of the first Gaussian thresholded mask,
6. n_{bgd_points}: the number of background point prompts to be sampled from the sure background region,
7. $d_{gap\_erosion}$: the kernel size for erosion used to create a gap between the foreground and background regions.

Figure A.3: Hyperparameters for SAM with automatic point prompts (SAM+pts)

Table A.1: Best TPE hyperparameter optimization results for CannyFill after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	23
$\sigma_{bilateral}$	21
$d_{closing}$	19
σ_{canny}	$8.7 \cdot 10^{-1}$

Table A.2: Best TPE hyperparameter optimization results for Gaussian after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	25
$\sigma_{bilateral}$	43
C	9
$d_{gaussian}$	39

Table A.3: Best TPE hyperparameter optimization results for SAM+pts after 100 trials.

Hyperparameter	Obtained value
$d_{bilateral}$	18
σ	141
C	5
$d_{gaussian}$	23
n_{fgd_pts}	250
n_{bgd_pts}	1352
$d_{gap\_erosion}$	5

A.1.2 Raw evaluation data

Table A.4: Raw Dice score data on easy images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	Gaussian	Otsu	SAM+pts-no-filter
HT10a	0.39	0.445	0.292	0.38
HT131a	0.152	0.464	0.401	0.343
HT17	0.253	0.367	0.157	0.389
HT21	0.191	0.402	0.22	0.278
HT22	0.226	0.382	0.319	0.32
HT7a	0.403	0.427	0.415	0.259
MA9	0.001	0.204	0.107	0.213
PH2	0.324	0.423	0.245	0.383
PH31a	0.009	0.354	0.343	0.338

Table A.5: Raw Dice score data on medium images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	Gaussian	Otsu	SAM+pts-no-filter
HT13	0.422	0.451	0.301	0.233
HT132	0.001	0.472	0.208	0.153
HT135a	0.107	0.343	0.249	0.223
HT154B	0	0.303	0.186	0.27
HT20	0.181	0.353	0.177	0.209
HT4	0.305	0.356	0.321	0.348
HT85b	0.317	0.293	0.231	0.287
HT90	0.188	0.266	0.251	0.311
PH31b	0.014	0.313	0.164	0.18

Table A.6: Raw Dice score data on hard images. Label column represents the name of the document segmented, while all other columns represent Dice scores obtained for those documents for a given model.

Label	CannyFill	Gaussian	Otsu	SAM+pts-no-filter
HT1	0.131	0.181	0.096	0.229
HT115b	0.059	0.19	0.09	0.124
HT122b	0.212	0.268	0.138	0.177
HT140	0.225	0.237	0.158	0.213
HT3	0.141	0.155	0.133	0.158
HT30	0.282	0.271	0.232	0.312
HT8a	0.295	0.402	0.182	0.231
HT9b	0.1	0.212	0.205	0.197
KN1a	0.02	0.3	0.228	0.334

A.1.3 Summary statistics

Table A.7: Summary Statistics per Model of Dice Score on all images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.183	0.132	0.025	27
Gaussian	0.327	0.093	0.018	27
Otsu	0.224	0.087	0.017	27
SAM+pts-no-filter	0.263	0.076	0.015	27

Table A.8: Summary Statistics per Model of Dice Score on easy images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.217	0.147	0.049	9
Gaussian	0.385	0.077	0.026	9
Otsu	0.278	0.105	0.035	9
SAM+pts-no-filter	0.322	0.061	0.02	9

Table A.9: Summary Statistics per Model of Dice Score on medium images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.171	0.154	0.051	9
Gaussian	0.35	0.07	0.023	9
Otsu	0.232	0.054	0.018	9
SAM+pts-no-filter	0.246	0.063	0.021	9

Table A.10: Summary Statistics per Model of Dice Score on hard images.

Model	Mean Dice	Standard deviation	Standard error	n
CannyFill	0.163	0.097	0.032	9
Gaussian	0.246	0.075	0.025	9
Otsu	0.162	0.053	0.018	9
SAM+pts-no-filter	0.22	0.068	0.023	9

A.2 Interactive tool development

A.2.1 Usability tests

A.2.1.1 Requirement analysis and first usability test

The 5th of May, a requirement analysis and first usability test of a MVP was performed with Ester through a video call. That MVP was composed of two tools: a classical method (Gaussian), and an interactive SAM version (MobileSAM). After trying out the tools, she gave the impression that she could probably use the outputs as draft drawings to be completed with a pen, which might save her some time.

For the classical baseline, the grainy resolution (due to the input image quality) bothered her a bit. The output mask was also noisy at times. And not all lines were always picked up.

For the SAM version, she pointed out that it did not pick up the noisy tablet damage as much as the classical method did. But she was worried that the interaction with SAM would take longer than simply doing strokes manually. The box-prompt workflow was not completely successful during the test. Due to bugs, boxes mostly did not work. and when they did, the segmented forms were not useful.

A.2.1.2 Second (small) usability test

The 8th of May, another usability test was performed on an updated MVP with Ester through a video call. Now, the box prompts worked, and

the MobileSAM was replaced with SAM2.1. The segmentation quality per box-prompted sign was noticeably better than previously. Together with Lukas Esterle, the conclusions were to make a slider for the classical model to decrease and increase granularity. It was also suggested that box prompts for SAM could be computationally preselected based on the mask from the classical model, but this was discarded as it was not deemed needed.

A.2.1.3 Third usability and final acceptance test

The 12th of May, a third usability test was conducted in person at AIAS in Århus, including the first part of the usability experiment. As all 45 images could not be drawn all at once, the .ora exports from the tool were saved, and finished at a later time in batches by Ester on her own, for each of them recording the time it took to do so. Due to the huge amount of time it took to process the images with the tool (especially with the SAM version), the experiment was stopped after just 3 images.

$\sqrt{17}$
$\Omega \int \delta e^{i\pi} = -1$
{2.7182818284}ϑφερτυθιοπσδφγηξκλ
χ^2
$\sum$
λ

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk